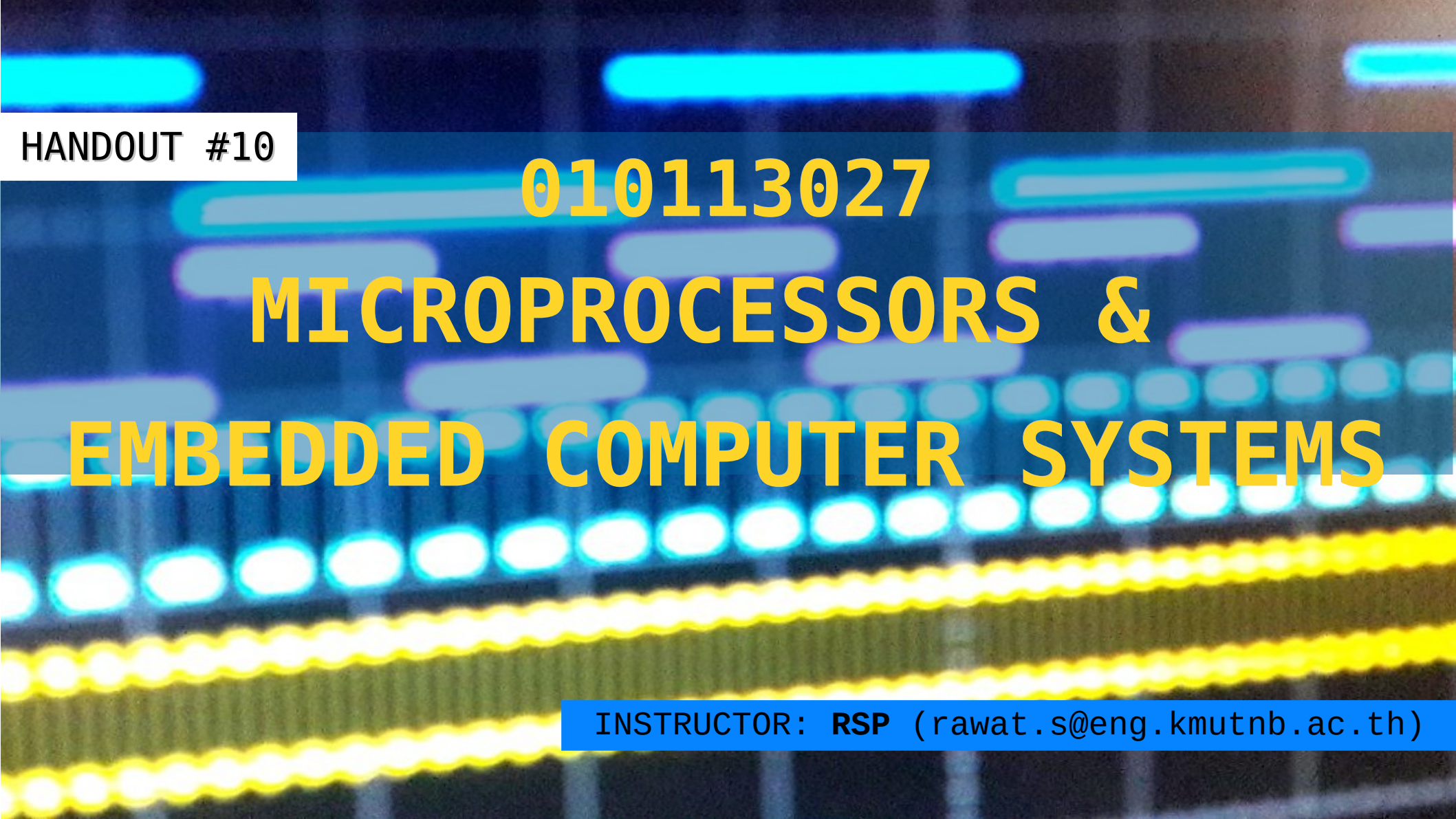


HANDOUT #10

010113027
MICROPROCESSORS & EMBEDDED COMPUTER SYSTEMS

INSTRUCTOR: **RSP** (rawat.s@eng.kmutnb.ac.th)

What We Will Learn...

- AVR Timer/Counter 1 (16-bit) and Operation Modes
- Differences between Timer/Counter1 and Timer/Counter 0/2
- ADC Sampling in Auto-Trigger Mode using Timer 1
- Frequency Measurement with Timer/Counter 1
- Pulse Counting using Timer/Counter 1 with Input Capture
- Periodic Pulse Test Signal Generation with Timers
- I/O Follower: Polling vs. Interrupt-driven Methods

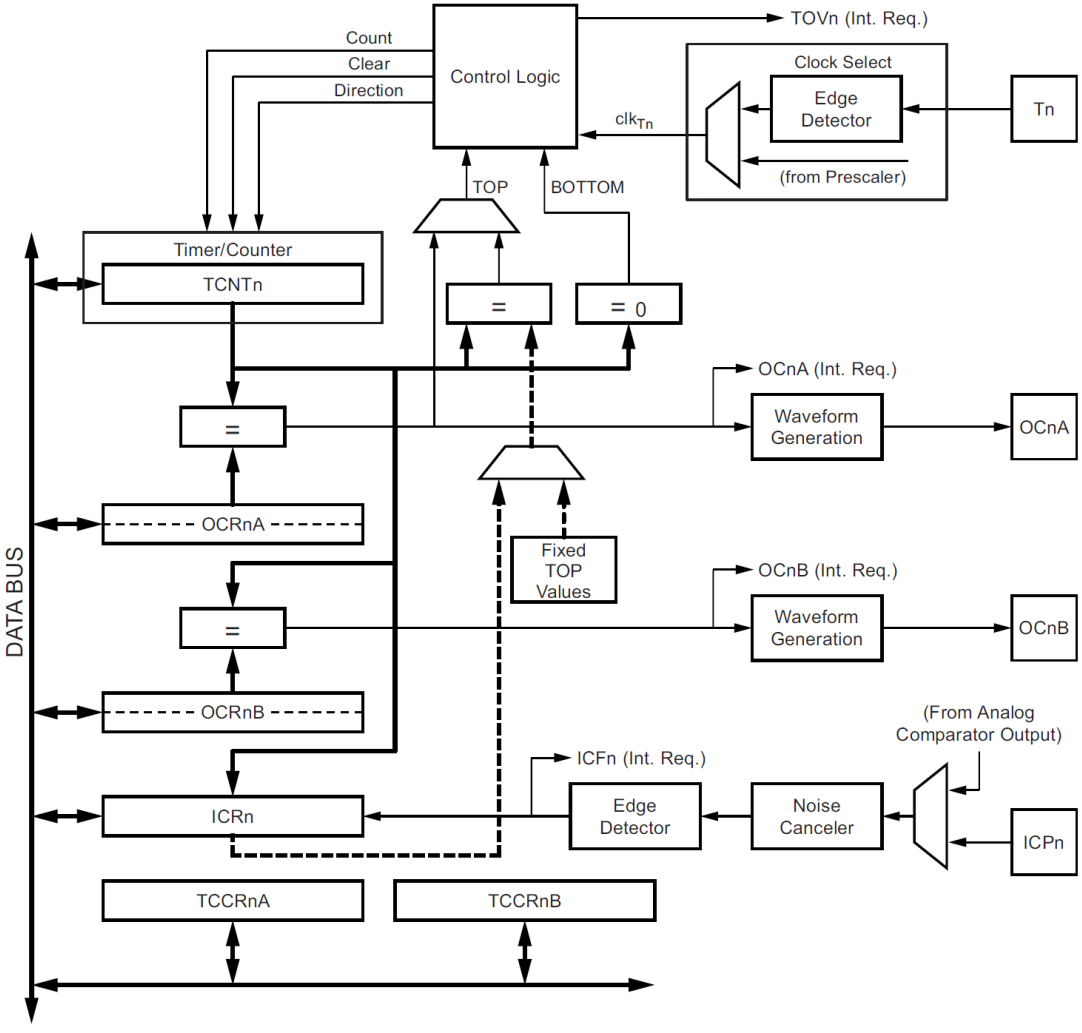
Timer1 Block Diagram

Timer1 Pins

- **T1** pin (input)
- **ICR** pin (input)
- **OC1A** pin (output)
- **OC1B** pin (output)

Timer1 Registers

- **TCNT1** (16-bit)
- **OCR1A** (16-bit)
- **OCR1B** (16-bit)
- **ICR1** (16-bit)
- **TCCR1A** (8-bit)
- **TCCR1B** (8-bit)
- **TCCR1C** (8-bit)



Timer1 vs. Timer0 & Timer2

Timer0 / Timer2

- **TCNTn/TCNTn**: 8-bit (0-255 counter range)
- **TCNTn**, **OCRnA**, **OCRnB**: 8-bit
- **PWM mode**: 8-bit only

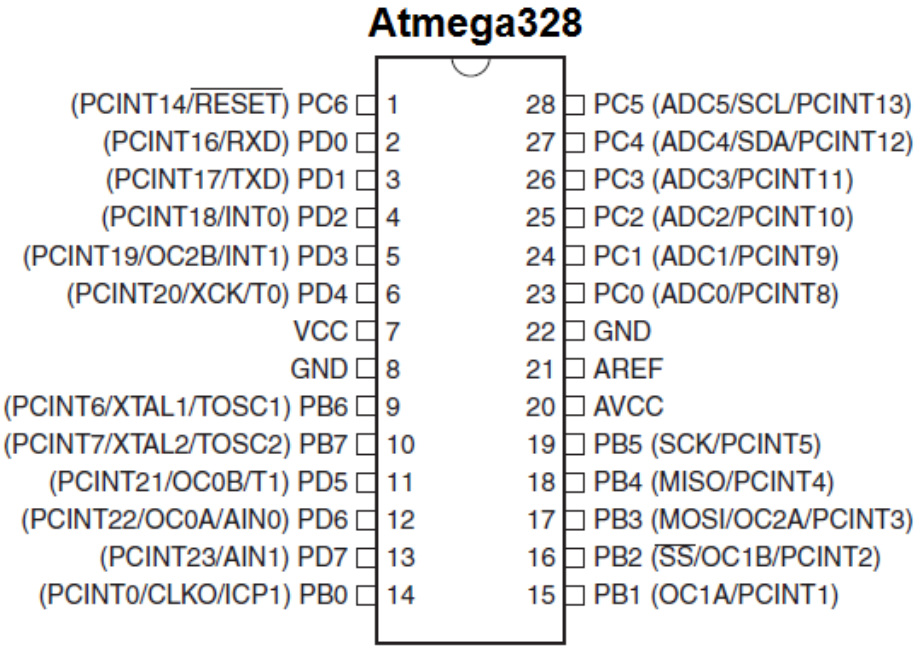
Timer1:

- **TCNT1**: 16-bit (0-65535 counter range)
- **TCNT1**, **OCR1A**, **OCR1B**: 16-bit
- **PWM mode**: 8-bit, 9-bit and 10-bit

Timer1 has an Input Capture unit.

- Input Capture pin (**ICP1**): **PB0/D8** pin
- **ICR1** register (16-bit)
- When enabled, an edge on the **ICP1** pin causes **TCNT1** to be latched into **ICR1**.
- **Applications**
  - Frequency or period measurement
  - Pulse width or duty cycle measurement

Timer0:	OC0A/PD6 (D6)	OC0B/PD5 (D5)
Timer1:	OC1A/PB1 (D9)	OC1B/PB2 (D10)
Timer2:	OC2A/PB3 (D11)	OC2B/PD3 (D3)



Timer ISR Functions

Timer0

- ISR(TIMER0_OVF_vect) { … } // Overflow
- ISR(TIMER0_COMPA_vect) { … } // Compare Match A
- ISR(TIMER0_COMPB_vect) { … } // Compare Match B

Timer1

- ISR(TIMER1_OVF_vect) { … } // Overflow
- ISR(TIMER1_COMPA_vect) { … } // Compare Match A
- ISR(TIMER1_COMPB_vect) { … } // Compare Match B
- ISR(TIMER1_CAPT_vect) { … } // Input Capture

Timer2

- ISR(TIMER2_OVF_vect) { … } // Overflow
- ISR(TIMER2_COMPA_vect) { … } // Compare Match A
- ISR(TIMER2_COMPB_vect) { … } // Compare Match B

Timer1 Registers

TCCR1A – Timer/Counter1 Control Register A

Bit	7	6	5	4	3	2	1	0	
(0x80)	COM1A1	COM1A0	COM1B1	COM1B0	–	–	WGM11	WGM10	TCCR1A
Read/Write	R/W	R/W	R/W	R/W	R	R	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

TCCR1B – Timer/Counter1 Control Register B

Bit	7	6	5	4	3	2	1	0	
(0x81)	ICNC1	ICES1	–	WGM13	WGM12	CS12	CS11	CS10	TCCR1B
Read/Write	R/W	R/W	R	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

TCCR1C – Timer/Counter1 Control Register C

Bit	7	6	5	4	3	2	1	0	
(0x82)	FOC1A	FOC1B	–	–	–	–	–	–	TCCR1C
Read/Write	R/W	R/W	R	R	R	R	R	R	
Initial Value	0	0	0	0	0	0	0	0	

TCNT1H and TCNT1L – Timer/Counter1

Bit	7	6	5	4	3	2	1	0	
(0x85)	TCNT1[15:8]								TCNT1H
(0x84)	TCNT1[7:0]								TCNT1L
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

TIMSK1 – Timer/Counter1 Interrupt Mask Register

Bit	7	6	5	4	3	2	1	0	
(0x6F)	–	–	ICIE1	–	–	OCIE1B	OCIE1A	TOIE1	TIMSK1
Read/Write	R	R	R/W	R	R	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

TIFR1 – Timer/Counter1 Interrupt Flag Register

Bit	7	6	5	4	3	2	1	0	
0x16 (0x36)	–	–	ICF1	–	–	OCF1B	OCF1A	TOV1	TIFR1
Read/Write	R	R	R/W	R	R	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

OCR1AH and OCR1AL – Output Compare Register 1 A

Bit	7	6	5	4	3	2	1	0	
(0x89)	OCR1A[15:8]								OCR1AH
(0x88)	OCR1A[7:0]								OCR1AL
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

OCR1BH and OCR1BL – Output Compare Register 1 B

Bit	7	6	5	4	3	2	1	0	
(0x8B)	OCR1B[15:8]								OCR1BH
(0x8A)	OCR1B[7:0]								OCR1BL
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Timer1 Registers

TCNT1 – 16-bit Counter Register

- **TCNT1** is a 16-bit counter register: **TCNT1H** (High byte) and **TCNT1L** (Low byte)
- The min and max counter values are: 0x0000 (**BOTTOM**) and 0xFFFF (**MAX**)

TCCR1A (Timer/Counter1 Control Register A)

- **COM1A[1:0]** – Compare Output Mode for Channel A
- **COM1B[1:0]** – Compare Output Mode for Channel B
- **WGM1[1:0]** – Waveform Generation Mode (lower bits)

TCCR1B (Timer/Counter1 Control Register B)

- **ICNC1** – Input Capture Noise Canceler
- **ICES1** – Input Capture Edge Select
- **WGM1[3:2]** – Waveform Generation Mode (higher bits)
- **CS1[2:0]** – Clock Select (prescaler selection)

TCCR1C (Timer/Counter1 Control Register C)

- **FOC1A** – Force Output Compare for Channel A
- **FOC1B** – Force Output Compare for Channel B (Used in non-PWM modes)

Timer1 Registers

OCR1A (Output Compare Register 1A)

- **OCR1A** is a 16-bit register, composed of **OCR1AH:OCR1AL**.
- Used to compare with the value of **TCNT1** to generate the output signal on the **OC1A** pin.

OCR1B (Output Compare Register 1B)

- **OCR1B** is also a 16-bit register, composed of **OCR1BH:OCR1BL**.
- Used to compare with **TCNT1** to generate the output signal on the **OC1B** pin.

TIMSK1 (Timer/Counter1 Interrupt Mask Register)

- **OCIE1A** – Output Compare A Match Interrupt Enable
- **OCIE1B** – Output Compare B Match Interrupt Enable
- **TOIE1** – Overflow Interrupt Enable
- **ICIE1** – Input Capture Interrupt Enable

TIFR1 (Timer/Counter1 Interrupt Flag Register)

- **TOV1** – Overflow Flag
- **OCF1A** – Output Compare A Match Flag
- **OCF1B** – Output Compare B Match Flag
- **ICF1** – Input Capture Flag

Timer1 Modes

Waveform Generation Mode Bit Description(1)

Mode	WGM13	WGM12 (CTC1)	WGM11 (PWM11)	WGM10 (PWM10)	Timer/Counter Mode of Operation	TOP	Update of OCR1X at	TOV1 Flag Set on
0	0	0	0	0	Normal	0xFFFF	Immediate	MAX
1	0	0	0	1	PWM, phase correct, 8-bit	0x00FF	TOP	BOTTOM
2	0	0	1	0	PWM, phase correct, 9-bit	0x01FF	TOP	BOTTOM
3	0	0	1	1	PWM, phase correct, 10-bit	0x03FF	TOP	BOTTOM
4	0	1	0	0	CTC	OCR1A	Immediate	MAX
5	0	1	0	1	Fast PWM, 8-bit	0x00FF	BOTTOM	TOP
6	0	1	1	0	Fast PWM, 9-bit	0x01FF	BOTTOM	TOP
7	0	1	1	1	Fast PWM, 10-bit	0x03FF	BOTTOM	TOP
8	1	0	0	0	PWM, phase and frequency correct	ICR1	BOTTOM	BOTTOM
9	1	0	0	1	PWM, phase and frequency correct	OCR1A	BOTTOM	BOTTOM
10	1	0	1	0	PWM, phase correct	ICR1	TOP	BOTTOM
11	1	0	1	1	PWM, phase correct	OCR1A	TOP	BOTTOM
12	1	1	0	0	CTC	ICR1	Immediate	MAX
13	1	1	0	1	(Reserved)	–	–	–
14	1	1	1	0	Fast PWM	ICR1	BOTTOM	TOP
15	1	1	1	1	Fast PWM	OCR1A	BOTTOM	TOP

Note: 1. The CTC1 and PWM11:0 bit definition names are obsolete. Use the WGM12:0 definitions. However, the functionality and location of these bits are compatible with previous versions of the timer.

Timer1 CTC Modes

In **CTC mode of Timer1**, there are two options for selecting the **TOP** value (16-bit):

1) OCR1A (CTC Mode 4)

- **TCNT1** counts up to **TOP** = **OCR1A**.
- When a **Compare Match A event** occurs, **TCNT1** is reset to 0 and a **Compare Match A interrupt** can be generated.
- Only one PWM output can be generated (**OC1B** only).

2) ICR1 (CTC Mode 12):

- **TCNT1** counts up to **TOP** = **ICR1**.
- When the compare match occurs, **TCNT1** is reset to 0.
- There is no "Input Capture interrupt" at **TOP**.
- Two PWM outputs of the same frequency can be generated (**OC1A** and **OC1B**).

Timer1 Fast-PWM Modes

Timer1 has several Fast PWM modes:

- The **TCNT1** counts from 0 to **TOP**, then resets immediately to 0.

1) Fixed PWM Frequency (fixed TOP value):

- **WGM13:0="0101"**: TOP = 0x00FF (8-bit)
- **WGM13:0="0110**": TOP = 0x01FF (9-bit)
- **WGM13:0="0111"**: TOP = 0x03FF (10-bit)

2) Adjustable PWM Frequency:

- **WGM13:0="1110"**: TOP = ICR1 (16-bit)
  - OCR1A and OCR1B define duty cycles
  - Frequency adjustable by changing ICR1
- **WGM13:0="1111**": TOP = OCR1A (16-bit)
  - OCR1A defines frequency, OCR1B defines duty.
  - Only OCR1B remains usable for PWM duty.

Timer1 Clock Selection: CS1[2:0] Bits in TCCR1A

Clock Select Bit Description

CS12	CS11	CS10	Description
0	0	0	No clock source (Timer/Counter stopped).
0	0	1	$clk_{I/O}$/1 (no prescaling)
0	1	0	$clk_{I/O}$/8 (from prescaler)
0	1	1	$clk_{I/O}$/64 (from prescaler)
1	0	0	$clk_{I/O}$/256 (from prescaler)
1	0	1	$clk_{I/O}$/1024 (from prescaler)
1	1	0	External clock source on T1 pin. Clock on falling edge.
1	1	1	External clock source on T1 pin. Clock on rising edge.

What is the longest timer interval of **Timer1 in Normal Mode**?

- Prescaler = 1024 (maximum)
- Counter runs from 0 → 65535 (16-bit overflow)
- T = 2^16 x 1024 / 16MHz = 4.194 seconds

Applications of Timer1

1) Periodic Task Execution via ISR

- CTC mode (Mode 4 or Mode 12) with Compare Match Interrupt or Normal mode (Mode 0) with overflow interrupt
- Real-time sampling loop
- Software timer or time base for scheduler
- Periodic Trigger Source for ADC

2) PWM Output Generation

- Fast PWM (Mode 14) or
- Phase-Correct / Phase & Frequency Correct (Mode 10)

3) Timer1 Input Capture

- Frequency measurement
- Pulse width measurement
- Duty cycle measurement
- External event timestamping
- Event Counting (External Clock Source via **T1/PD5 pin**.

C Code Examples

- **Code Example 1:**
  - This example uses **Timer1 in Normal mode** to generate periodic events at a **2 Hz** rate.
  - The **Timer1 overflow ISR** reloads **TCNT1** with a preload value and toggles the **PB5** pin.
- **Code Example 2**:
  - This example uses **Timer1 in CTC mode** to generate periodic events at a **2 Hz** rate.
  - The T**imer1 compare match ISR** toggles the **PB5** pin.
- **Code Example 3:**
  - **Timer1** operates in **Fast PWM Mode 14 (WGM13:0 = 14).**
  - **ICR1** is used as **TOP** to set the **PWM** frequency.
  - The **OCR1A / OCR1B** registers control the duty cycles of the PWM outputs.

C Code Example 1: Timer1 in Normal Mode

```
#ifndef F_CPU
#define F_CPU 16000000UL
#endif
#include <avr/io.h>
#include <avr/interrupt.h>

// Preload value for Timer1 in Normal Mode
#define TIMER1_PRELOAD (65535 - F_CPU/1024/2)

void gpio_init() {
  DDRB |= (1<<PB5); // Set PB5 as output
}
void timer1_init(void) {
  // Stop timer, Normal mode (default)
  TCCR1A = TCCR1B = 0x00;
  TCNT1 = TIMER1_PRELOAD; // Preload counter
  // Enable overflow interrupt
  TIMSK1 |= (1<<TOIE1);
  // Set the prescaler = 1024 and start Timer1
  TCCR1B |= (1<<CS12) | (1<<CS10);
}
```

```
// Timer1 Overflow ISR
ISR(TIMER1_OVF_vect) {
  TCNT1 = TIMER1_PRELOAD; // Reload TCNT1
  PINB = (1<<PB5); // Toggle PB5
}

int main(void) {
  gpio_init();   // Initialize GPIO
  timer1_init(); // Initialize Timer1
  sei(); // Enable global interrupts
  while (1) {}
}
```

16MHz/ 1024 / 2 = 7812.5 → 7812

16MHz / 1024 / 7812 = ~2Hz

Toggle Rate: 2 Hz

Toggle Interval: 500 ms

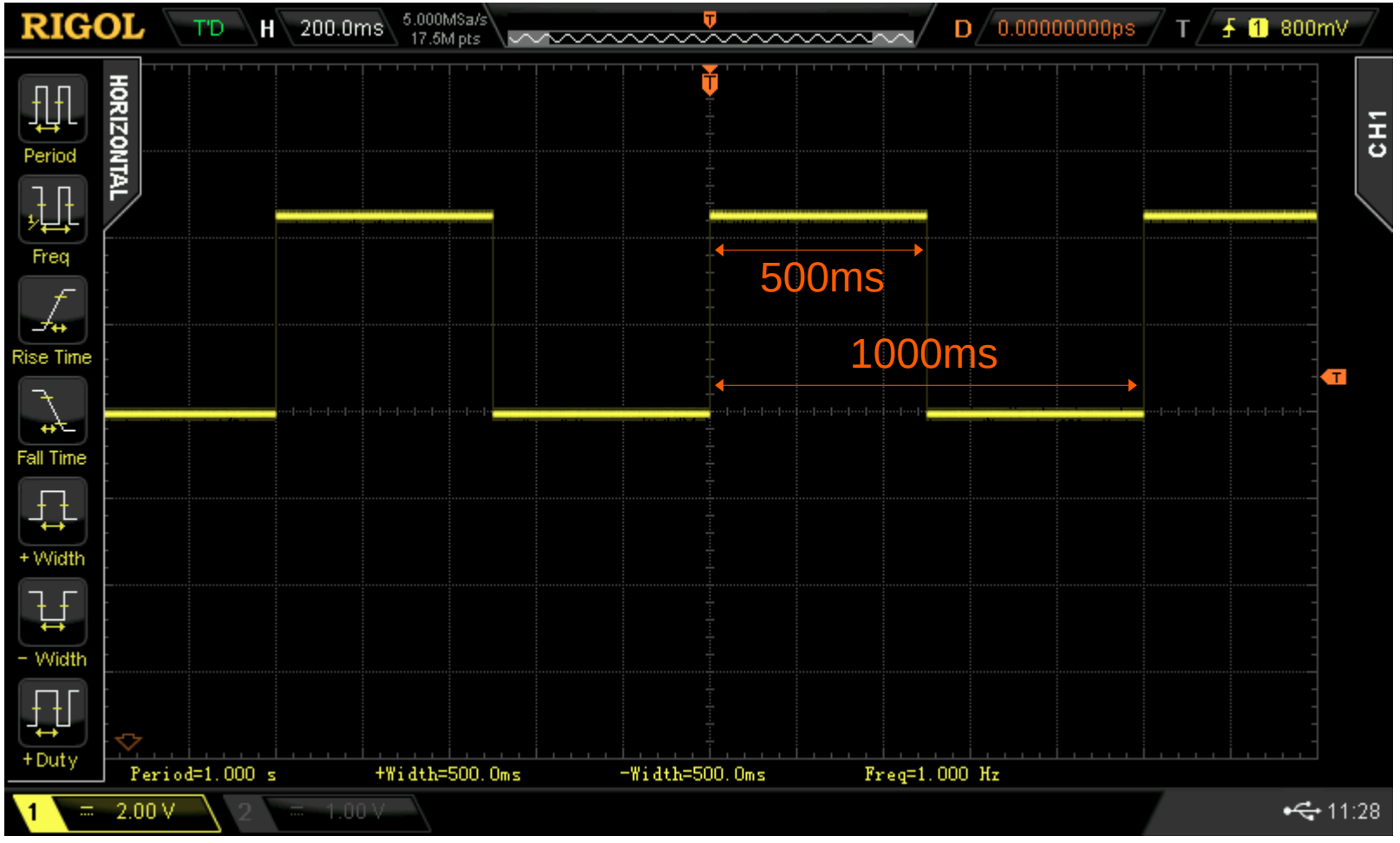
PB5 frequency: 1Hz

C Code Example 2: Timer1 in CTC Mode

```
#ifndef F_CPU
#define F_CPU 16000000UL
#endif
#include <avr/io.h>
#include <avr/interrupt.h>

#define OCR1A_VALUE (F_CPU/1024/2 - 1)

void gpio_init(void) {
  DDRB |= (1<<PB5); // Set PB5 as output
}
void timer1_init(void) {
  TCCR1A = TCCR1B = 0x00; // Stop Timer1
  // Set compare match A value
  OCR1A = OCR1A_VALUE;
  TCCR1B |= (1<<WGM12); // Use CTC mode
  // Enable Compare A interrupt
  TIMSK1 |= (1<<OCIE1A);
  // Set the prescaler = 1024, start Timer1
  TCCR1B |= (1<<CS12)| (1<<CS10);
}
```

```
// Timer1 Compare Match A ISR
ISR(TIMER1_COMPA_vect) {
  PINB = (1<<PB5); // Toggle PB5
}

int main(void) {
  gpio_init();   // Initialize GPIO
  timer1_init(); // Initialize Timer1
  sei(); // Enable global interrupts
  while (1) {}
}
```

Waveform Capture with DSO

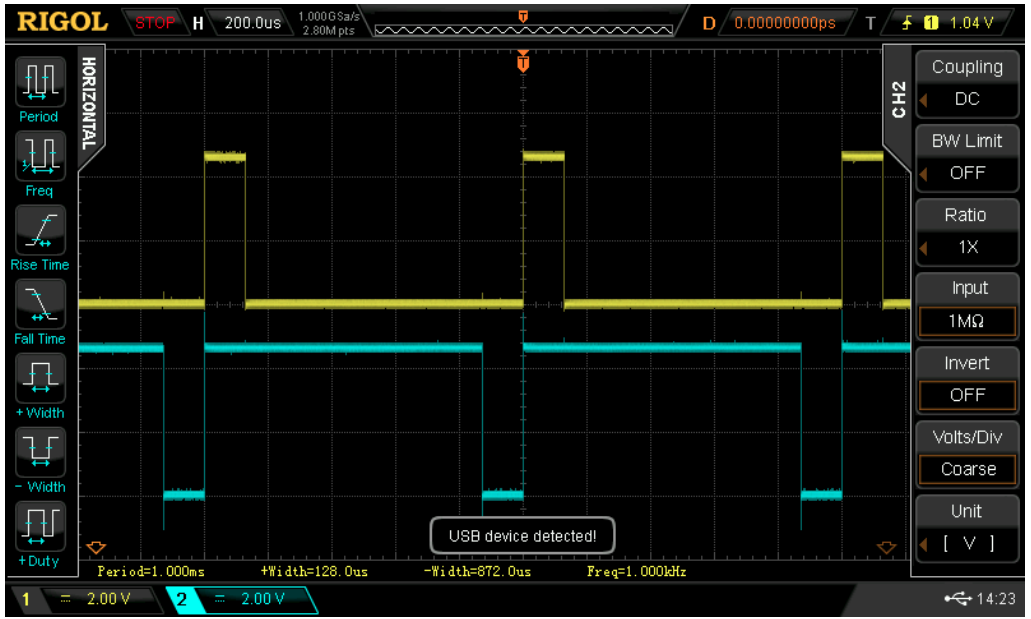


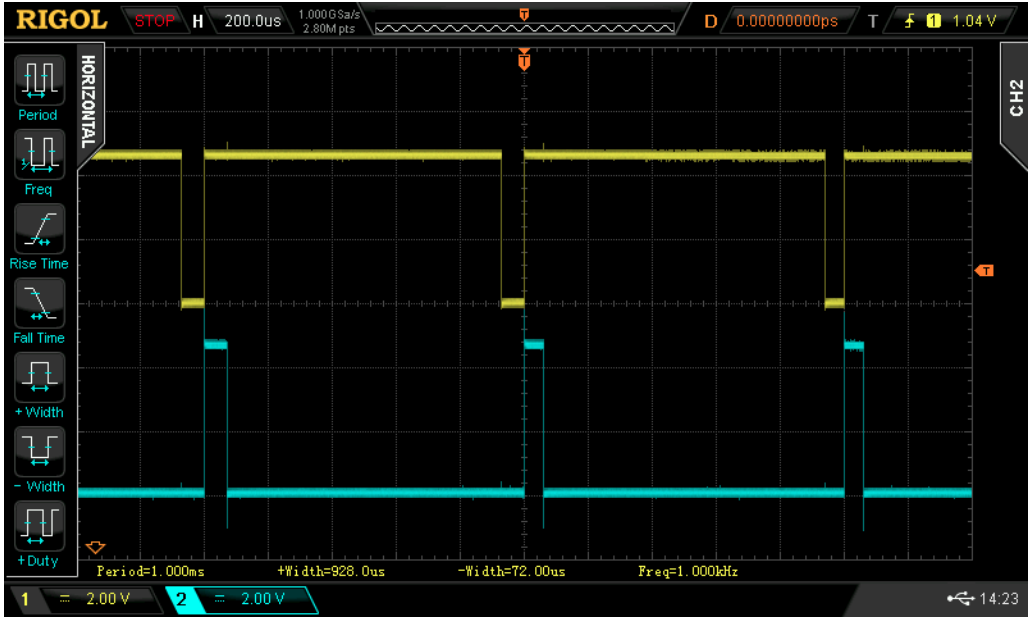
C Code Example 3: Fast PWM Mode 14

```
#ifndef F_CPU
#define F_CPU 16000000UL
#endif
#include <avr/io.h>
#include <util/delay.h>
#define PWM_FREQ  1000UL
#define PRESCALE  8

#define TOP_VALUE \
        ((F_CPU/(PRESCALE*PWM_FREQ))-1)

void gpio_init() {
  // OC1A/PB1 & OC1B/PB2 as output
  DDRB |= (1<<PB1)| (1<<PB2);
}

void update_pwm_duty(uint8_t percent) {
  uint16_t value =
           ((uint32_t)percent*ICR1)/100;
  OCR1A = value;
  OCR1B = ICR1 - value;
}
```

```
void timer1_init(void) {
  TCCR1A = TCCR1B = 0x00; // Stop Timer1
  // Fast PWM Mode 14
  TCCR1A |= (1<<COM1A1)| (1<<COM1B1)| (1<<WGM11);
  TCCR1B |= (1<<WGM13) | (1<<WGM12);
  ICR1  = TOP_VALUE;   // Set period (TOP)
  OCR1A = TOP_VALUE/2; // 50% duty initially
  OCR1B = TOP_VALUE/2;
  // Set the prescaler = 8, start Timer1
  TCCR1B |= (1<<CS11);
}
int main(void) {
  gpio_init();
  timer1_init();
  while (1) {
    for (uint8_t dc=0; dc<=100; dc++) {
      update_pwm_duty(dc);
      _delay_ms(10);
    }
  }
}
```

Waveform Capture with DSO





Timer1 Compare Output Mode

Timer1 output pins (ATmega328P):

- OC1A / PB1 (Arduino D9)
- OC1B / PB2 (Arduino D10)

Configuration bits in TCCR1A:

- COM1A1:0 bits for OC1A
- COM1B1:0 bits for OC1B

COM1x1	COM1x0	Meaning
0	0	Normal port operation (OC1A/OC1B disconnected)
0	1	Toggle on compare match (non-PWM mode only)
1	0	Clear on compare match A or B (for PWM mode)
1	1	Set on compare match A or B (for PWM mode)

Modes

- Normal (non-PWM) mode
- Fast PWM mode
- Phase-Correct / Frequency-Correct PWM mode

PWM Modes

- Non-PWM mode (Normal and CTC) vs. PWM mode (Fast-PWM & Phase-Correct).

Output Non-inverting vs. Inverting on Compare Match

- Non-inverting PWM: Set at BOTTOM and clear on compare match
- Inverting PWM: Clear at BOTTOM and set on compare match

Timer1 Compare Output Mode

Compare Output Mode, non-PWM

COM1A1/COM1B1	COM1A0/COM1B0	Description
0	0	Normal port operation, OC1A/OC1B disconnected.
0	1	Toggle OC1A/OC1B on compare match.
1	0	Clear OC1A/OC1B on compare match (set output to low level).
1	1	Set OC1A/OC1B on compare match (set output to high level).

Compare Output Mode, Fast PWM

COM1A1/COM1B1	COM1A0/COM1B0	Description
0	0	Normal port operation, OC1A/OC1B disconnected.
0	1	WGM13:0 = 14 or 15: Toggle OC1A on compare match, OC1B disconnected (normal port operation). For all other WGM1 settings, normal port operation, OC1A/OC1B disconnected.
1	0	Clear OC1A/OC1B on compare match, set OC1A/OC1B at BOTTOM (non-inverting mode)
1	1	Set OC1A/OC1B on compare match, clear OC1A/OC1B at BOTTOM (inverting mode)

Compare Output Mode, Phase Correct and Phase and Frequency Correct PWM

COM1A1/COM1B1	COM1A0/COM1B0	Description
0	0	Normal port operation, OC1A/OC1B disconnected.
0	1	WGM13:0 = 9 or 11: Toggle OC1A on compare match, OC1B disconnected (normal port operation). For all other WGM1 settings, normal port operation, OC1A/OC1B disconnected.
1	0	Clear OC1A/OC1B on compare match when up-counting. Set OC1A/OC1B on compare match when down counting.
1	1	Set OC1A/OC1B on compare match when up-counting. Clear OC1A/OC1B on compare match when down counting.

C Code Examples

- **Code Example 4:**
  - **Timer1** is used in Normal mode (Mode 0, **WGM13:0=0000**) with Compare Output Toggle.
  - **TOP** is **65535.**
  - **COM1A1:0 = "01"** and **COM1B1:0 = "01"**:
    - **OC1A** toggles whenever **TCNT1 == OCR1A**.
    - **OC1B** toggles whenever **TCNT1 == OCR1B**.
    - **OCR1A=OCR1B=0:** Toggle once per full 16-bit cycle.
- **Code Example 5**:
  - **Timer1** operates in **Fast PWM Mode 14** (**WGM13:0=1110**).
  - **TOP** is defined by **ICR1**.
  - **OC1A** and **OC1B** are used to generate PWM outputs with adjustable duty cycles.
  - **OC1A**: **Non-inverting PWM** (set at **BOTTOM**, clear on compare match).
  - **OC1B**: **Inverting PWM** (clear at **BOTTOM**, set on compare match).

C Code Examples

Code Example 6:

- **Timer1** is configured to operate in **Phase & Frequency Correct PWM mode**, with **TOP** = **ICR1**.
- The PWM waveform is generated using dual-slope (up–down) counting, resulting in a **symmetric (center-aligned) waveform**.
- **Non-inverting PWM mode** is used on both **OC1A** and **OC1B** outputs.

Code Example 7:

- This example implements an active-LOW **PLC-style TON (Timer ON Delay)** using **Timer1** on **ATmega328P**.
- **PD2** is configured as an input with an internal pull-up resistor.
- **PB5** is configured as a digital output.
- **Timer1** operates in **CTC mode** to generate a 1 ms interrupt.
- When **PD2** stays LOW continuously for T_ON=0.5 sec (500 ticks), the output (**PB5**) turns ON.
- If the input goes HIGH before T_ON, the timer resets and the output remains OFF.

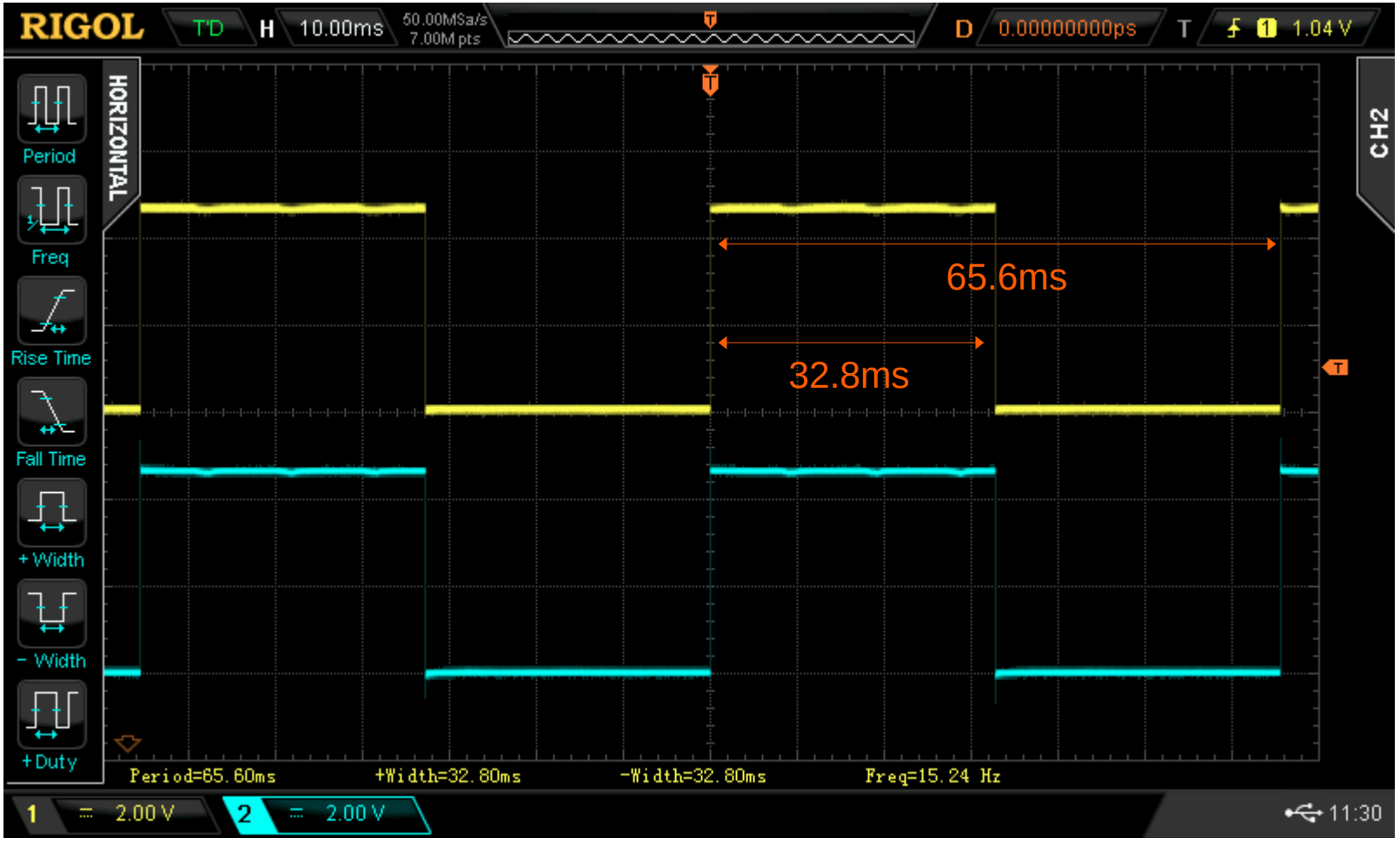
C Code Example 4: Timer1 with OC1A/OC1B Output (Normal Mode)

```c
#include <avr/io.h>

void gpio_init() {
  // Set OC1A and OC1B pins as output
  DDRB |= (1<<PB1) | (1<<PB2);
}

// 32.8msec Toggle rate
void timer1_init(void) {
  // Stop Timer 1 (Normal Mode 0, WGM=0000)
  TCCR1A = TCCR1B = 0;
  OCR1A = 0;
  OCR1B = 0;
  // Output toggles every time if TCNT1 == OCR1x
  TCCR1A |= (1<<COM1A0) | (1<<COM1B0);
  // Set prescaler = 8 and start Timer1
  TCCR1B |= (1<<CS11);
}
```

```c
int main(void) {
  gpio_init();   // Initialize GPIO
  timer1_init(); // Initialize Timer1
  while (1) {}
}
```

Toggle Rate = 16MHz/8/65536
= 30.51756Hz
Toggle Interval = 32.768 ms

Waveform Capture with DSO



C Code Example 5: Timer1 with OC1A/OC1B Output (Fast-PMM)

```c
#include <avr/io.h>

void gpio_init() {
  // Set OC1A and OC1B pins as output
  DDRB |= (1<<PB1) | (1<<PB2);
}

#define TOP   (40000)
// PWM: OC1A 25%, OC1B 25% (inverted)
void timer1_init(void) {
  TCCR1A = TCCR1B = 0;  // Stop Timer 1
  // Set ICR1 (TOP), OCR1A and OCR1B
  ICR1 = TOP-1; OCR1A = TOP/4-1; OCR1B = TOP/4-1;
  // Fast PWM Mode 14 (WGM=1110)
  TCCR1A |= (1<<WGM11);
  TCCR1B |= (1<<WGM13) | (1<<WGM12);
  // Clear on compare match A (non-inverting)
  //   Set on compare match B (inverting)
  TCCR1A |= (1<<COM1A1)|(1<<COM1B1)|(1<<COM1B0);
  // Set prescaler = 8 and start Timer1
  TCCR1B |= (1<<CS11);
}
```

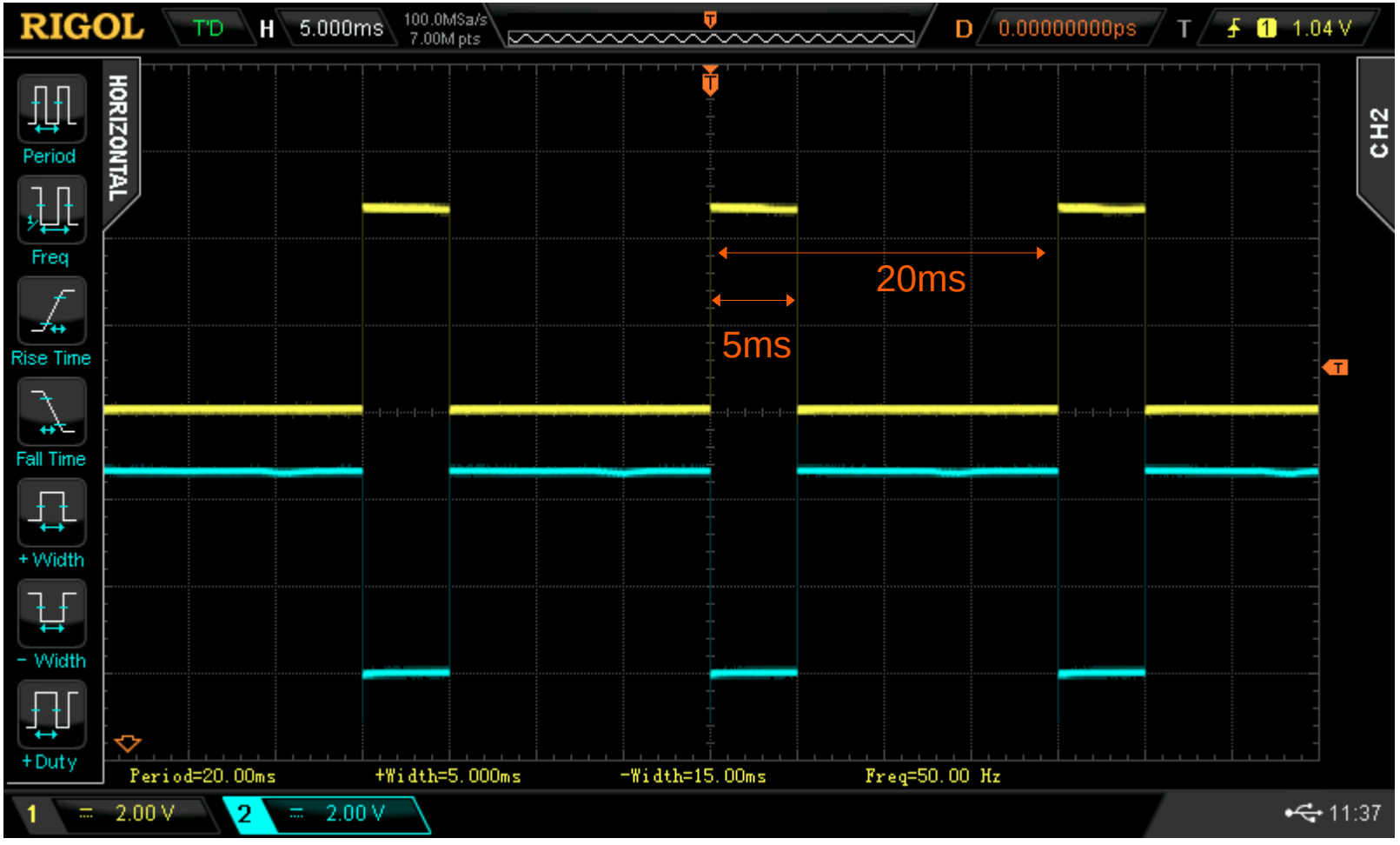
```c
int main(void) {
  gpio_init();   // Initialize GPIO
  timer1_init(); // Initialize Timer1
  while (1) {}
}
```

PWM Freq. = 16MHz/8/40000 = 50Hz

OC1A Duty Cycle = 25% (5 ms)

OC1B Duty Cycle = 25% (inverted)

Waveform Capture with DSO



C Code Example 6: Timer1 with OC1A/OC1B Output (Phase-Correct PMM)

```c
#include <avr/io.h>

#define TOP 2000

// Freq. 500Hz, Phase-correct PWM, non-inverted
void timer1_init(void) {
  TCCR1A = TCCR1B = 0;  // Stop Timer 1
  ICR1  = TOP-1;
  OCR1A = TOP/2; // OC1A 50%
  OCR1B = TOP/4; // OC1B 25%
  TCCR1A |= (1<<WGM11);
  TCCR1B |= (1<<WGM13); // WGM13:0=1010
  // OC1A/OC1B output (non-inverted)
  TCCR1A |= (1<<COM1A1)|(1<<COM1B1);
  // Set prescaler = 8 and start Timer1
  TCCR1B |= (1<<CS11);
}
```

```c
int main(void) {
  gpio_init();   // Initialize GPIO
  timer1_init(); // Initialize Timer1
  while (1) {}
}
```

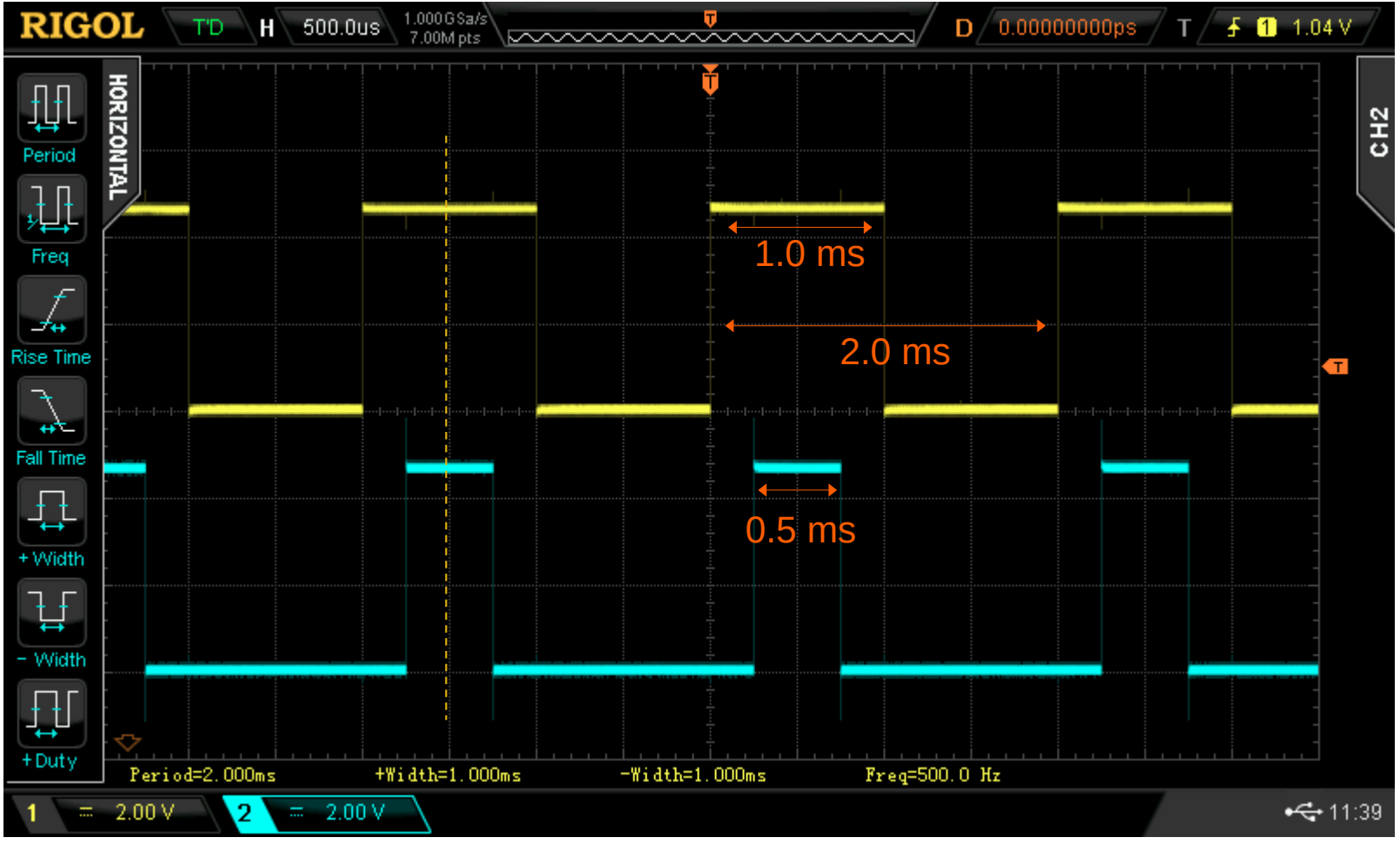
PWM Freq. = 16MHz/8/2000/2 = 500Hz

Period = 2ms

OC1A Duty Cycle = 50% (1.0ms)

OC1B Duty Cycle = 25% (0.5ms)

Waveform Capture with DSO

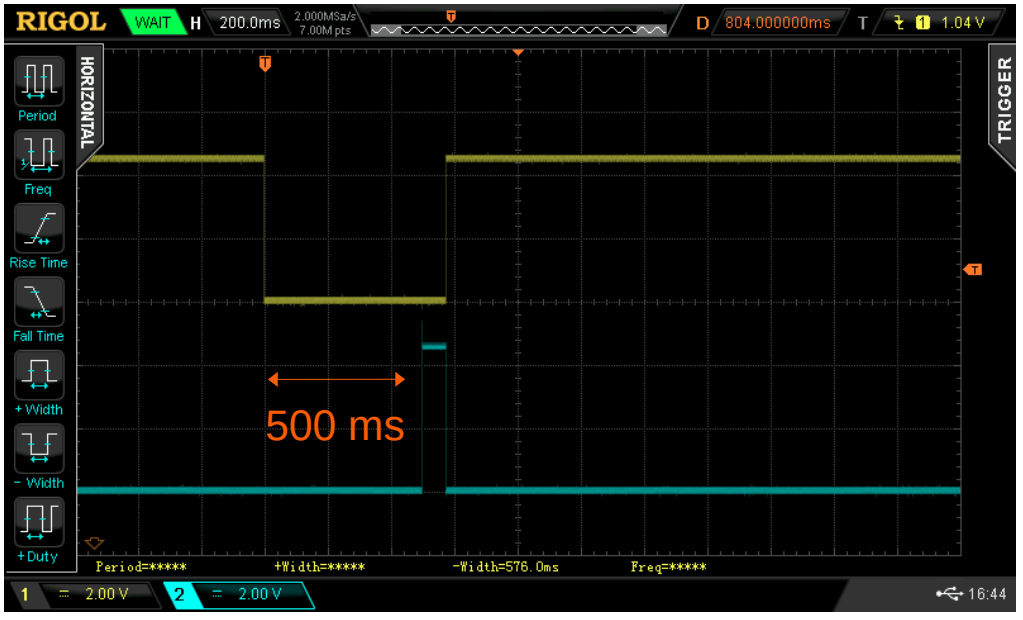


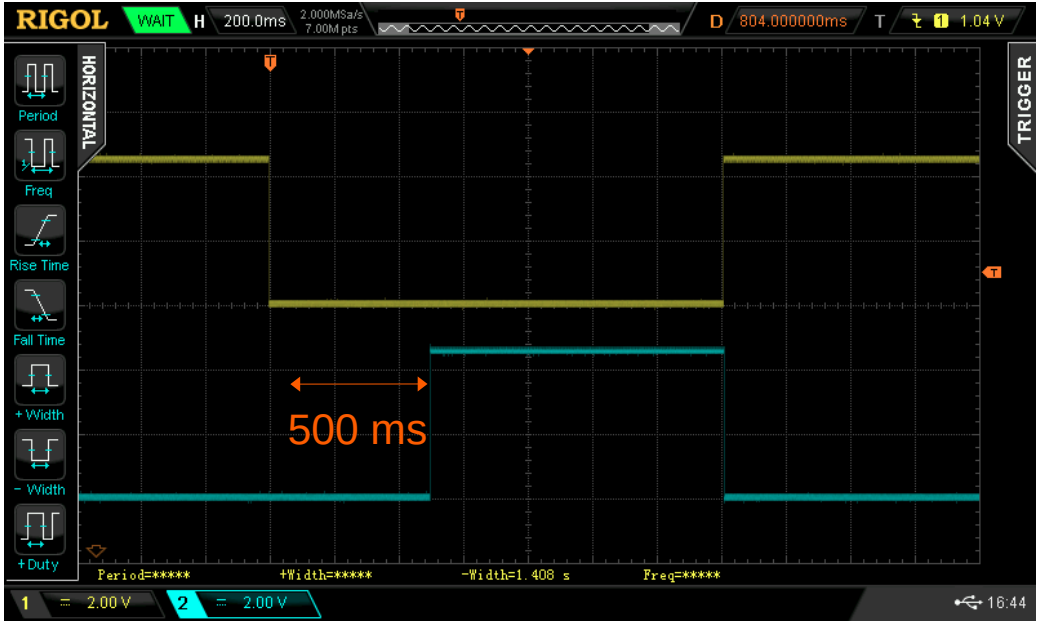
C Code Example 7: ON-delay Timer

```c
#include <avr/io.h>
#include <avr/interrupt.h>
#include <stdint.h>
#include <stdbool.h>

#define DELAY_MS 500

volatile uint16_t ms_counter = 0;

void gpio_init(void) {
  DDRD  &= ~(1<<PD2); // PD2 input
  PORTD |=  (1<<PD2); // Internal pull-up
  DDRB  |=  (1<<PB5); // PB5 output
  PORTB &= ~(1<<PB5); // Output OFF
}

void timer1_init(void) {
  TCCR1A = TCCR1B = 0;
  TCCR1B |= (1<<WGM12); // CTC mode
  OCR1A  = 249;         // Set period
  TIMSK1 |= (1<<OCIE1A);
  // Set the prescaler = 64, start Timer1
  TCCR1B |= (1<<CS11)|(1<<CS10);
}
```

```c
ISR(TIMER1_COMPA_vect) {
  if (!(PIND & (1<<PD2))) { // PD2 low
    if (ms_counter < DELAY_MS)
      ms_counter++;
    else
      PORTB |= (1<<PB5);  // Output ON
  }
  else {
    ms_counter = 0;
    PORTB &= ~(1<<PB5);  // Output OFF
  }
}

int main(void) {
  gpio_init();
  timer1_init();
  sei();
  while (1) {}
}
```

Waveform Capture with DSO





C Code Examples

Code Example 8:

- This code uses two hardware timers: **Timer1** and **Timer2**.
- **Timer2** operates in **CTC mode** and generates an interrupt every 1 ms. Within the **Timer2 interrupt service routine**, **Timer1** is started.
- **Timer1** is configured in **CTC mode** with the **OC1A (PB1) pin** set to toggle automatically on compare match. This hardware toggling generates a burst of five output pulses without software intervention.

Code Example 9:

- This code uses **Timer1 in Normal mode with Input Capture enabled** to count pulses arriving on the **ICP1** pin.
- **Timer2** is configured in **CTC mode** to generate **a burst of pulses on the OC2A pin**, producing a square wave at the specified frequency.
- Each rising edge detected on the **ICP1 pin** triggers the **Timer1 Input Capture interrupt**, and the ISR increments a pulse counter. **Timer1** counts the number of incoming pulses generated by **Timer2**.

C Code Example 8: Burst-Mode Pulse Generation

```
#ifndef F_CPU
#define F_CPU 16000000UL
#endif

#include <avr/io.h>
#include <avr/interrupt.h>
#include <stdint.h>

#define FREQ_HZ  (200000UL)
#define PRESCALE (1UL)
#define OCR1A_VAL \
        ((F_CPU/(2UL*PRESCALE*FREQ_HZ))-1)

static volatile uint16_t toggle_count = 0;

void gpio_init(void) {
  DDRB  |=  (1<<PB1); // OC1A output
  PORTB &= ~(1<<PB1); // Output LOW
}
```

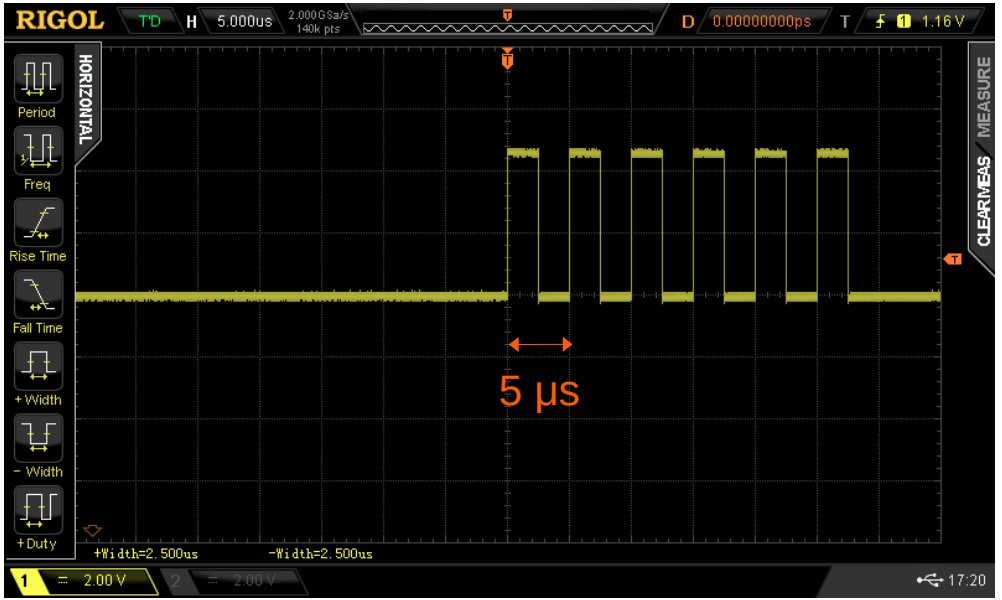
```
void timer2_init(void) {
  TCCR2A = TCCR2B = 0;
  // CTC mode
  TCCR2A |= (1<<WGM21);
  OCR2A = 249; // Period = 1 ms
  TIMSK2 |= (1<<OCIE2A);
  // Set the prescaler=64, start Timer2
  TCCR2B |= (1<<CS22);
}

ISR(TIMER2_COMPA_vect) {
  // Start new burst every 1 ms
  toggle_count = 2*5;
  TCNT1 = 0;
  TIFR1 = (1<<OCF1A);
  TIMSK1 |= (1<<OCIE1A);
  // Start Timer1 (prescaler = 1)
  TCCR1B = (1<<WGM12) | (1<<CS10);
}
```

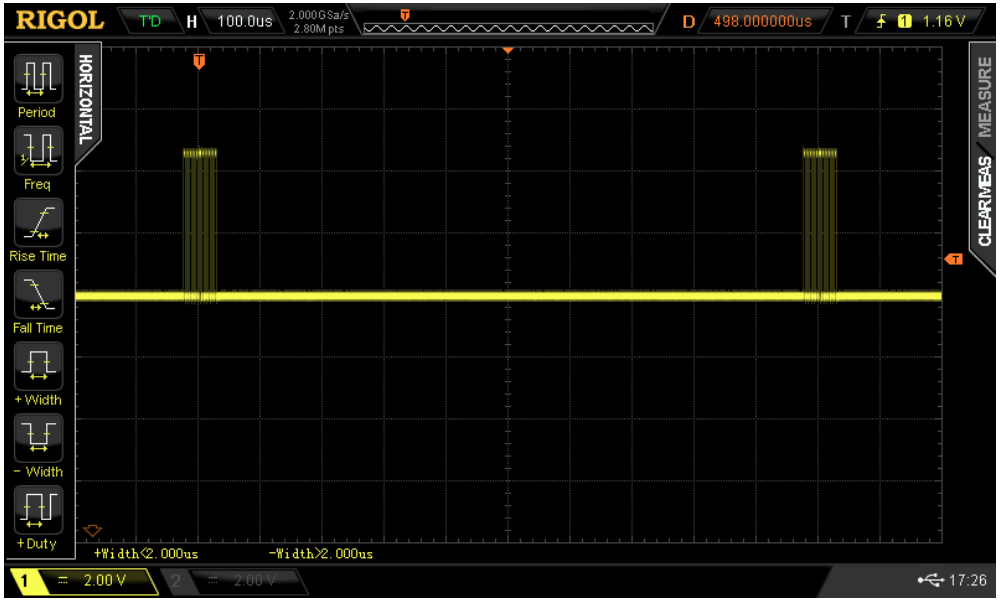
C Code Example 8: Burst-Mode Pulse Generation

```
void timer1_init(void) {
  TCCR1A = TCCR1B = 0;
  OCR1A  = OCR1A_VAL;
  // Toggle OC1A on compare match A
  TCCR1A |= (1<<COM1A0);
  // CTC mode
  TCCR1B |= (1<<WGM12);
}

ISR(TIMER1_COMPA_vect) {
  if (--toggle_count == 0) {
   // Stop Timer1
   TCCR1B &= ~((1<<CS12)|(1<<CS11)|(1<<CS10));
   // Disable Timer1 interrupt
   TIMSK1 &= ~(1<<OCIE1A);
  }
}
```

```
int main(void) {
  gpio_init();
  timer1_init();
  timer2_init();
  sei();
  while (1) {}
}
```

Waveform Capture with DSO





C Code Example 9: Pulse Counting with Timer1 with Input Capture

```c
#ifndef F_CPU
#define F_CPU 16000000UL
#endif
#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/delay.h>
#include <stdint.h>
#include <stdint.h>
#include <stdio.h>

#define FREQ_HZ 50000UL  // Pulse frequency
#define TIM2_PRESCALE 8UL
#define OCR2A_VAL \
   ((F_CPU/(2UL*TIM2_PRESCALE*FREQ_HZ)) - 1)

extern void usart_init(void);
extern void usart_print(const char *s);

volatile uint16_t toggle_count = 0;
volatile uint8_t  burst_done = 0;
volatile uint32_t pulse_counter = 0;
```

```c
void gpio_init(void) {
  // OC2A output (PB3 / Arduino D11)
  DDRB |= (1<<PB3);
  // ICP1 input (PB0 / Arduino D8)
  DDRB &= ~(1<<PB0);
}
// Timer1 in Normal mode
void timer1_init(void) {
  TCCR1A = TCCR1B = 0;
  TCNT1 = 0;
  // Rising edge trigger for ICP1
  TCCR1B |= (1<<ICES1);
  // Enable input capture interrupt
  TIMSK1 |= (1<<ICIE1);
  // Prescaler = 1 (full resolution)
  TCCR1B |= (1<<CS10);
}

ISR(TIMER1_CAPT_vect) {
  pulse_counter++;
}
```

C Code Example 9: Pulse Counting with Timer1

```
void timer2_init(void) {
  TCCR2A = TCCR2B = 0;
  TCNT2  = 0;
  OCR2A  = (uint8_t)OCR2A_VAL;
  // CTC mode
  TCCR2A |= (1<<WGM21);
  // Toggle OC2A on compare
  TCCR2A |= (1<<COM2A0);
}
ISR(TIMER2_COMPA_vect) {
  if (--toggle_count == 0) {
    // Stop Timer2 clock
    TCCR2B = 0;
    // Disable Timer1 OC2A interrupt
    TIMSK2 &= ~(1<<OCIE2A);
    burst_done = 1;
  }
}
```

```
void emit_pulses(uint16_t N) {
  cli();  // Disable global interrupts
  toggle_count = 2*N; // 1 pulse = 2 toggles
  burst_done = 0;
  TCNT2 = 0;
  TIFR2 = (1<<OCF2A);
  TIMSK2 |= (1<<OCIE2A);
  // Start Timer2 with prescaler
#if (TIM2_PRESCALE == 1)
  TCCR2B = (1<<CS20);
#elif (TIM2_PRESCALE == 8)
  TCCR2B = (1<<CS21);
#elif (TIM2_PRESCALE == 64)
  TCCR2B = (1<<CS22);
#endif
  sei(); // Enable global interrupts
}
```

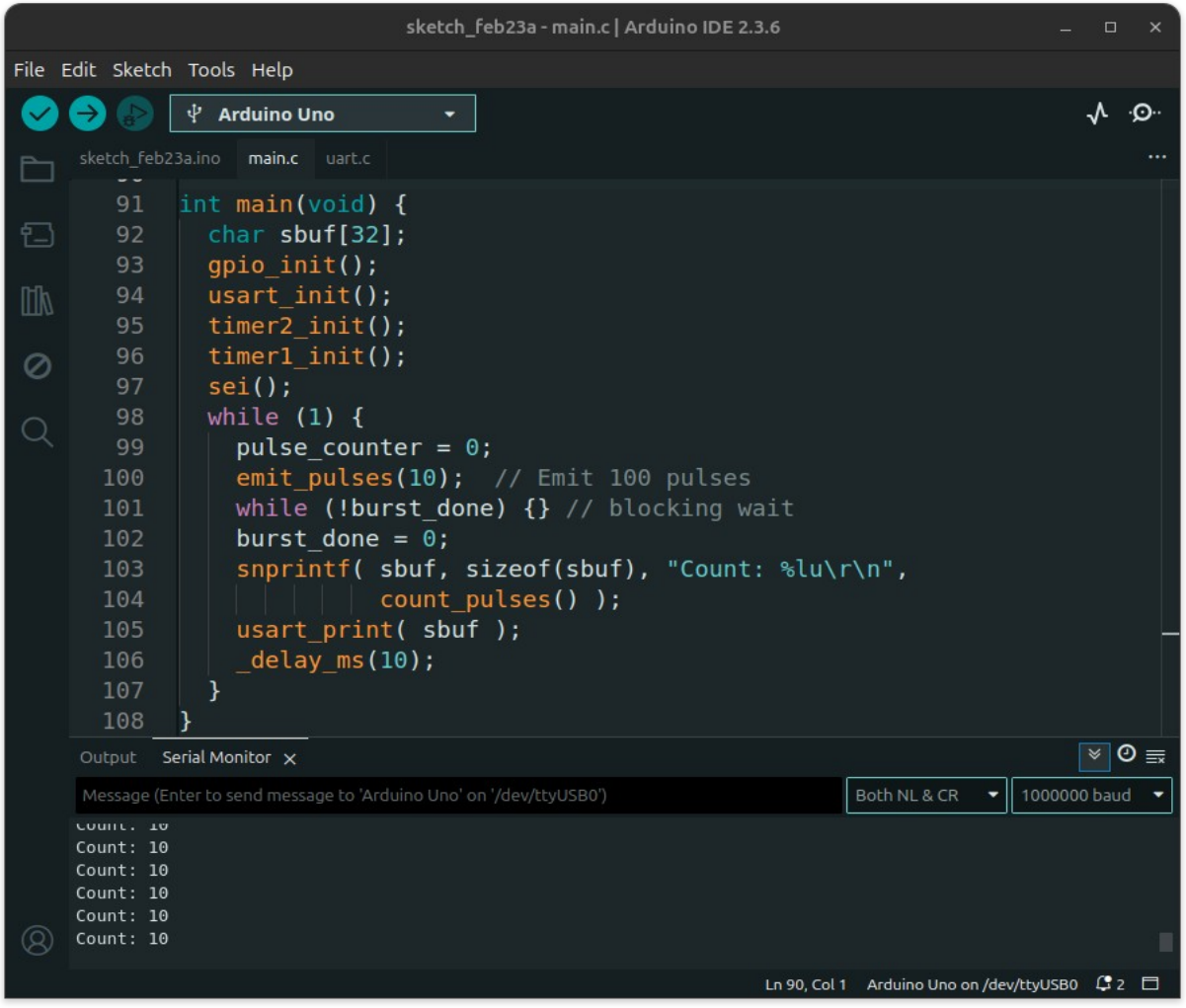
C Code Example 9: Pulse Counting with Timer1

```
uint32_t count_pulses(void) {
  uint32_t pulses;
  cli();
  pulses = pulse_counter;
  sei();
  return pulses;
}
```

- **Timer2** is used to generate pulses.
- **Timer1** is configured to count these pulses.
- A rising edge on the **ICP1** pin triggers the **Timer1 Input Capture interrupt** if **ICES1** is enabled.
- The **OC2A/PB3 (D11) output pin** (**PB3** / D11) is connected to the **Timer1** external clock input **PB0** / **ICP1 (D8)** using a jumper wire.

```
int main(void) {
  char sbuf[32];
  gpio_init();
  usart_init();
  timer2_init();
  timer1_init();
  sei();
  while (1) {
    pulse_counter = 0;
    emit_pulses(10);  // Emit pulses
    while (!burst_done) {} // Blocking wait
    burst_done = 0;
    snprintf( sbuf, sizeof(sbuf),
              "Count: %lu\r\n",
              count_pulses() );
    usart_print( sbuf );
    _delay_ms(10);
  }
}
```

Arduino Serial Monitor



C Code Examples

Code Example 10:

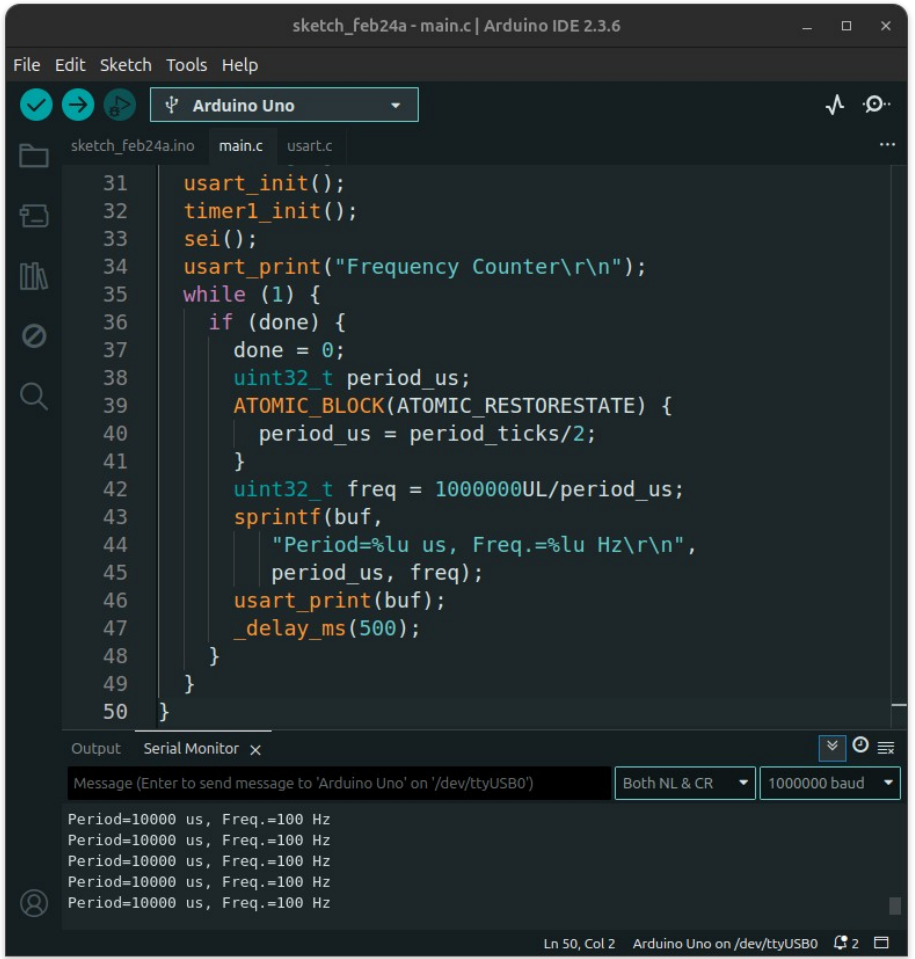
- This code uses **Timer1 (Normal Mode)** with **Input Capture** enabled.
- The time resolution is **0.5 µs**, and the maximum measurable interval before a timer overflow occurs is 32.768 ms (corresponding to ~30.5 Hz).
- Since **Timer1** is only 16-bit, timer overflow may occur when measuring low-frequency input signals.

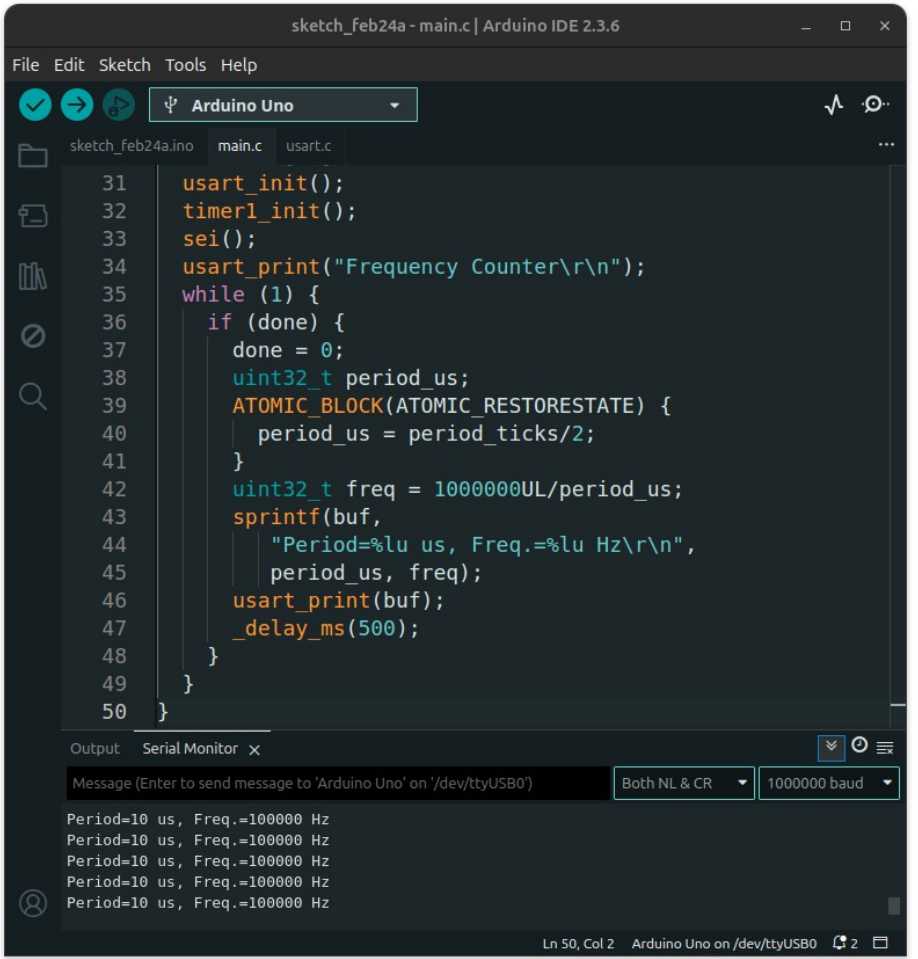
C Code Example 10: High Frequency Measurement with Timer1

```c
#define F_CPU 16000000UL
#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/delay.h>
#include <util/atomic.h>
#include <stdio.h>

extern void usart_init(void);
extern void usart_print(const char *s);

volatile uint16_t last = 0, period_ticks = 0;
volatile uint8_t  done = 0;

ISR(TIMER1_CAPT_vect) {
  uint16_t now = ICR1; // Read ICR1
  period_ticks = now - last;
  last = now;
  done = 1;
}
void timer1_init(void) {
  DDRB &= ~(1<<DDB0);  // ICP1 (PB0/D8) input
  TCCR1A = 0;
  TCCR1B = (1<<ICES1)  // Rising edge capture
         | (1<<CS11);  // Set prescaler=8
  TIMSK1 = (1<<ICIE1); // Enable interrupt
}
```

```c
int main(void) {
  char buf[40];
  usart_init();
  timer1_init();
  sei();
  usart_print("Frequency Counter\r\n");
  while (1) {
    if (done) {
      done = 0;
      uint32_t period_us;
      ATOMIC_BLOCK(ATOMIC_RESTORESTATE) {
        period_us = period_ticks/2;
      }
      uint32_t freq = 1000000UL/period_us;
      sprintf(buf,
         "Period=%lu us, Freq.=%lu Hz\r\n",
         period_us, freq);
      usart_print(buf);
      _delay_ms(500);
    }
  }
}
```

Arduino Serial Monitor





C Code Example 11: Frequency Measurement with Timer1/Timer2

```c
#define F_CPU 16000000UL

#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/atomic.h>
#include <stdbool.h>
#include <stdio.h>

void usart_init(void);
void usart_print(const char *s);

volatile uint32_t pulse_count = 0;
volatile uint16_t ovf_count = 0;
volatile uint16_t countdown = 0;
volatile bool done = false;

void gpio_init() {
  DDRD  &= ~(1<<DDD5);  // D5 input
  PORTD &= ~(1<<PORTD5);
}
```

```c
void timer_start(uint16_t duration_ms) {
  uint8_t s = SREG;
  cli();
  done = false;
  ovf_count = 0;
  countdown = duration_ms;
  TCCR1A = TCCR1B = 0; TCNT1 = 0;
  TCCR2A = TCCR2B = 0; TCNT2 = 0;
  OCR2A  = 124;           // 1 ms
  TCCR2A |= (1<<WGM21);  // CTC
  // Enable Timer2 compare match A interrupt
  TIMSK2 = (1<<OCIE2A);
  // Enable Timer1 overflow interrupt
  TIMSK1 = (1<<TOIE1);
  // T1=Clk source, rising edge, start Timer1
  TCCR1B = (1<<CS12)| (1<<CS11)| (1<<CS10);
  // Set Timer2 prescaler=128, start Timer2
  TCCR2B |= (1<<CS22) | (1<<CS20);
  SREG = s;
}
```

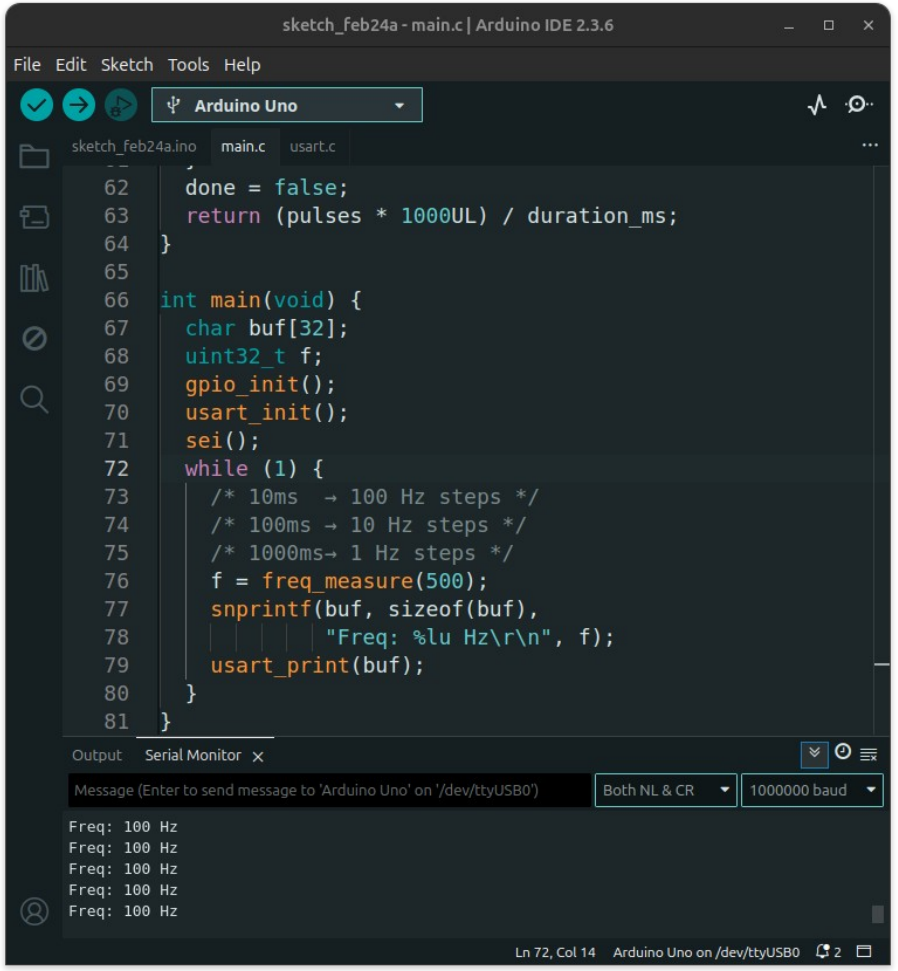
C Code Example 11: Frequency Measurement with Timer1/Timer2

```
ISR(TIMER1_OVF_vect) {
  ovf_count++;
}
ISR(TIMER2_COMPA_vect) {
  if (--countdown == 0) {
    TCCR1B = 0; // Stop Timer1
    pulse_count  = TCNT1;
    pulse_count |= (uint32_t)ovf_count<<16;
    TIMSK2 = 0; // Stop timer2
    done = true;
  }
}
uint32_t freq_measure(uint16_t duration_ms) {
  timer_start(duration_ms);
  while (!done) {} // Blocking wait
  uint32_t pulses;
  ATOMIC_BLOCK(ATOMIC_RESTORESTATE) {
    pulses = pulse_count;
  }
  done = false;
  return (pulses * 1000UL) / duration_ms;
}
```

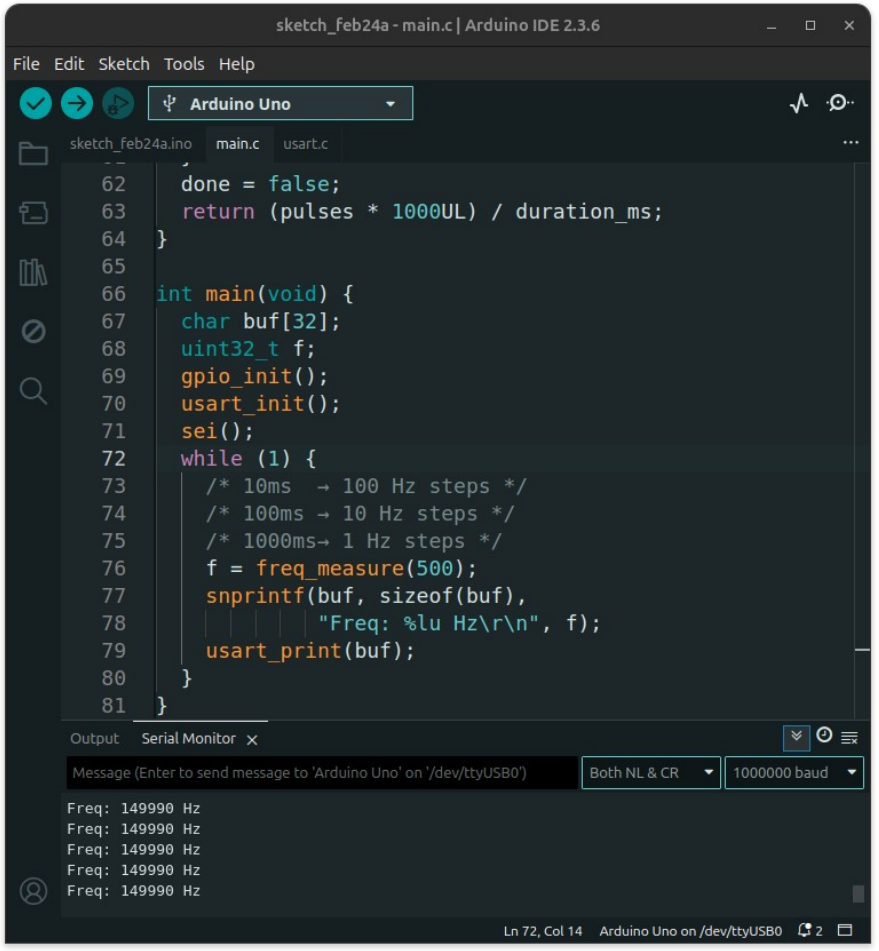
```
int main(void) {
  char buf[32];
  uint32_t f;
  gpio_init();
  usart_init();
  sei();
  while (1) {
    /* 10ms   → 100 Hz steps */
    /* 100ms  →  10 Hz steps */
    /* 1000ms →   1 Hz steps */
    f = freq_measure(500);
    snprintf(buf, sizeof(buf),
             "Freq: %lu Hz\r\n", f);
    usart_print(buf);
  }
}
```

Arduino Serial Monitor

100Hz test signal



150kHz test signal



C Code Example 12: Timer1 in Normal Mode + ADC Auto-Trigger

- This example demonstrates how to use **Timer/Counter1 in Normal (Overflow) mode** to automatically triggers the ADC for periodic sampling of **analog input ADC0 (PC0/A0)**.
- **Timer1** runs with a **prescaler of 8** and is preloaded on every overflow so that the **overflow rate is 1 kHz**.
- The **Timer1 overflow event** is selected as the **ADC Auto-Trigger source**, causing the ADC to start a conversion automatically at each **Timer1 overflow**.
- The **ADC conversion** complete interrupt reads the sample and signals the main loop to transmit the value through **USART**.
- An **LED on PB5** is driven HIGH at the start of each sampling period (timer overflow) and LOW when the **ADC conversion** finishes, allowing the sampling and conversion timing to be observed on an oscilloscope.

C Code Example 12: Timer1 in Normal Mode + ADC Auto-Trigger

```c
#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/atomic.h>
#include <stdio.h>

extern void usart_init(void);
extern void usart_print(const char *s);

#define PERIOD     (2000 - 1)
#define LOAD_VALUE (65535 - PERIOD)

volatile uint16_t adc_value = 0;
volatile uint8_t valid = 0;

void gpio_init() {
  DDRB |= (1<<PB5); // PB5 as output
}
```

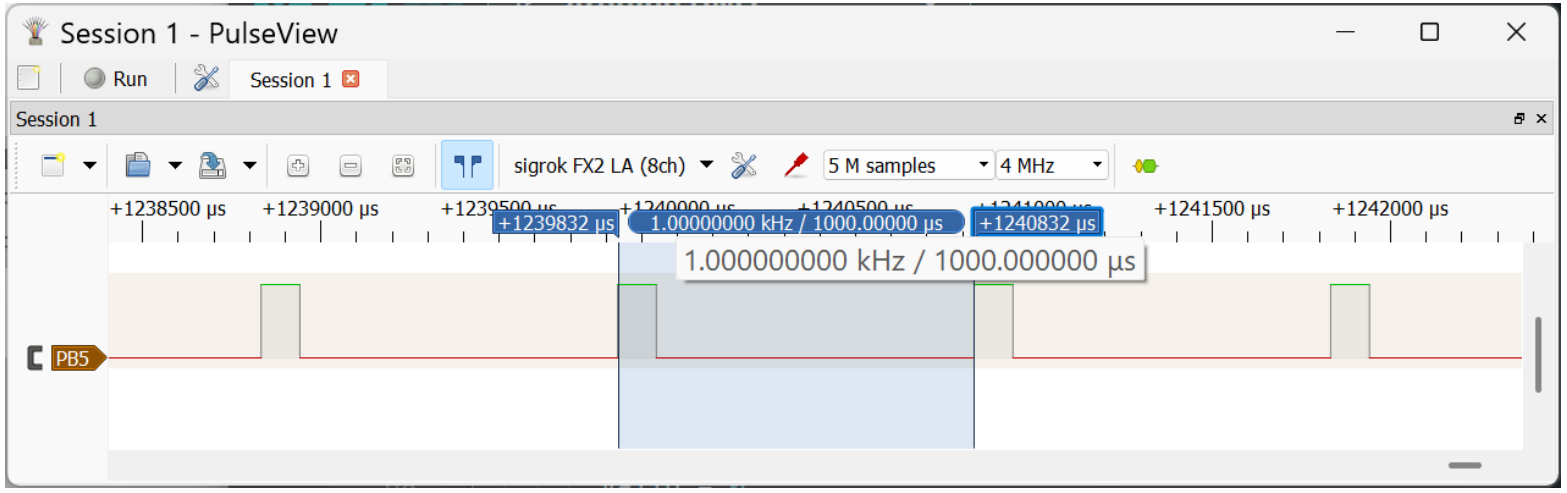
```c
ISR(TIMER1_OVF_vect) {
  TCNT1 = LOAD_VALUE;  // Reload Timer1 count
  PORTB |= (1<<PB5); // PB5 high
}

// Timer1 rate = 16MHz/8/2000 = 1000Hz
void timer1_init() {
  TCCR1A = TCCR1B = 0; // Use Normal mode
  // Load the initial value for Timer1 count
  TCNT1 = LOAD_VALUE;
  // Clear Timer1 overflow interrupt flag
  TIFR1 |= (1<<TOV1);
  // Enable Timer1 overflow interrupt
  TIMSK1 |= (1<<TOIE1);
  // Set the prescaler=8 and start Timer1
  TCCR1B |= (1<<CS11);
}
```

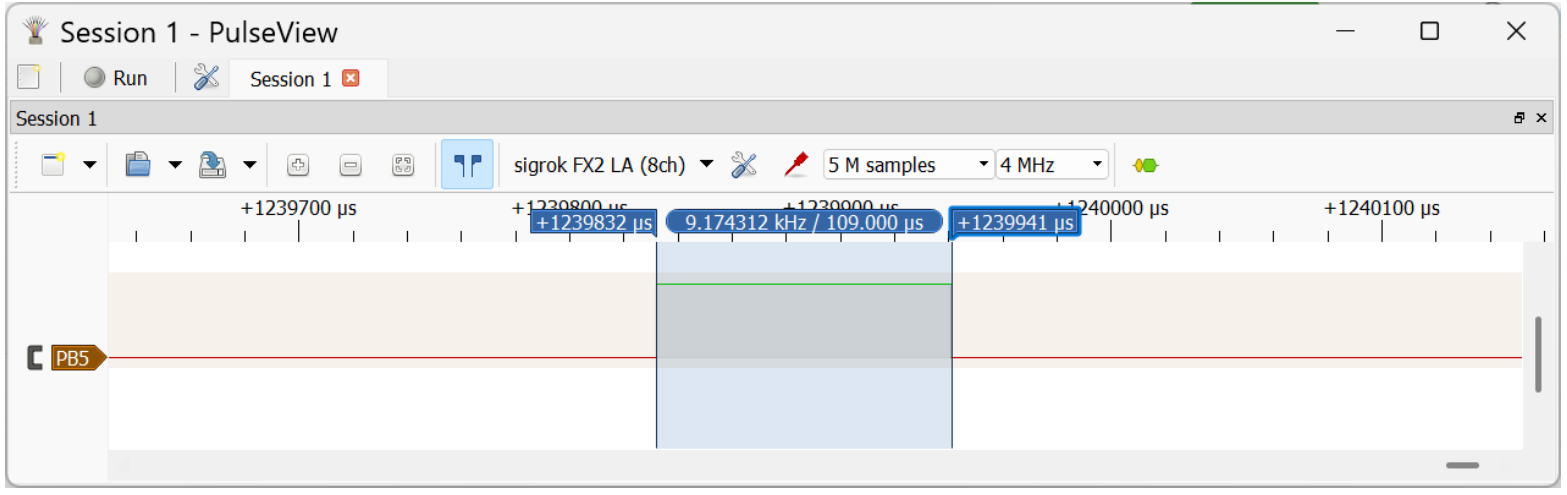
C Code Example 12: Timer1 in Normal Mode + ADC Auto-Trigger

```c
void adc_init() {
  // Set PC0/A0 as an input pin
  DDRC &= ~(1<<DDC0);
  // Disable Digital Input Buffer on A0
  DIDR0 |= (1<<ADC0D);
  // Set reference voltage to AVCC
  ADMUX |= (1<<REFS0);
  // Auto-Trigger Source: Timer1 Overflow
  ADCSRB = (1<<ADTS2)| (1<<ADTS1);
  // Enable ADC and set prescaler = 128
  ADCSRA = (1<<ADPS2)| (1<<ADPS1)| (1<<ADPS0)
         | (1<<ADIE) | (1<<ADEN) | (1<<ADATE);
}
// ISR for ADC conversion complete
ISR(ADC_vect) {
  adc_value = ADC;    // Save the ADC value
  valid = 1;          // Set sample valid flag
  PORTB &= ~(1<<PB5); // PB5 low
}
```

```c
int main(void) {
  char buf[8];
  uint16_t value;
  gpio_init();   // Initialize GPIO
  usart_init();  // Initialize USART
  adc_init();    // Initialize ADC
  timer1_init(); // Initialize Timer1
  sei();  // Enable global interrupts
  while (1) {
    if (valid) {
      ATOMIC_BLOCK(ATOMIC_RESTORESTATE) {
        valid = 0;
        value = adc_value;
      }
      snprintf(buf, sizeof(buf),
               "%u\r\n", value);
      usart_print(buf);
    }
  }
  return 0;
}
```

USB Logic Analyzer: Waveform Capture





C Code Example 13: Timer1 in CTC Mode + ADC Auto-Trigger

- In contrast to the previous code, this code configures **Timer1** in **CTC (Clear Timer on Compare Match) mode** to generate a **1 kHz sampling rate** using the **Compare Match B event**.
- Each compare match event automatically triggers the ADC in **auto-trigger mode**, enabling **periodic analog sampling**.
- The ADC continuously samples **ADC0 (PC0)** with **AVcc as the reference voltage**. When a conversion completes, the ADC interrupt stores the 10-bit result and sets a flag.
- The main loop safely reads the value using an atomic block and transmits it via USART.
- An LED on PB5 toggles during the timer interrupt to provide a timing/debug signal that indicates the 1 kHz trigger rate.

C Code Example 13: Timer1 in CTC Mode + ADC Auto-Trigger

```
#ifndef F_CPU
#define F_CPU 16000000UL
#endif

#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/atomic.h>
#include <stdio.h>

extern void usart_init(void);
extern void usart_print(const char *s);

volatile uint16_t adc_value = 0;
volatile uint8_t valid = 0;

#define PERIOD 250

void gpio_init() {
  DDRB |= (1<<PB5); // PB5 as output
}
```

```
ISR(TIMER1_COMPB_vect) {
  PINB = (1<<PB5); // Toggle LED
}

// Timer1 in CTC mode, 16MHz/64/250 = 1000Hz
void timer1_init(void) {
  // CTC mode (TOP = OCR1A)
  TCCR1A = 0;
  TCCR1B = (1<<WGM12);
  OCR1A = PERIOD-1; // Set the period (TOP)
  OCR1B = 0;
  // Set the prescaler = 64 and start Timer1
  TCCR1B |= (1<<CS11)| (1<<CS10);
}
```

C Code Example 13: Timer1 in CTC Mode + ADC Auto-Trigger

```c
void adc_init(void) {
  // PC0/ADC0 as input
  DDRC &= ~(1<<DDC0);
  // Disable digital input buffer for ADC0
  DIDR0 |= (1<<ADC0D);
  ADCSRB = ADCSRB = 0;
  ADMUX  = (1<<REFS0);  // Use AVcc as Vref
  // 0b101 = Timer1 Compare Match B
  ADCSRB |= (1<<ADTS2)| (1<<ADTS0);
  // Enable Timer1 Compare Match B interrupt
  TIMSK1 |= (1<<OCIE1B);
  ADCSRA |= (1<<ADEN) | (1<<ADATE)| (1<<ADIE)
         |  (1<<ADPS2)| (1<<ADPS1)| (1<<ADPS0);
  ADCSRA |= (1<<ADSC);  // Start a conversion
}

// ISR for ADC conversion complete
ISR(ADC_vect) {
  adc_value = ADC; // Save the ADC value
  valid = 1;       // Set sample valid flag
  PORTB &= ~(1<<PB5); // PB5 low
}
```

```c
int main(void) {
  char buf[8];
  uint16_t value;
  gpio_init();    // Initialize GPIO
  usart_init();   // Initialize USART
  adc_init();     // Initialize ADC
  timer1_init();  // Initialize Timer1
  sei();   // Enable global interrupts
  while (1) {
    if (valid) {
      ATOMIC_BLOCK(ATOMIC_RESTORESTATE) {
        valid = 0;
        value = adc_value;
      }
      snprintf(buf, sizeof(buf),
               "%u\r\n", value);
      usart_print(buf);
    }
  }
  return 0;
}
```

Arduino Serial Plotter

100Hz test signal (sinusoidal, Vpp=4V, voffset=2.5V), Sampling rate=1kHz

500Hz test signal (sinusoidal, Vpp=4V, voffset=2.5V), Sampling rate=10kHz

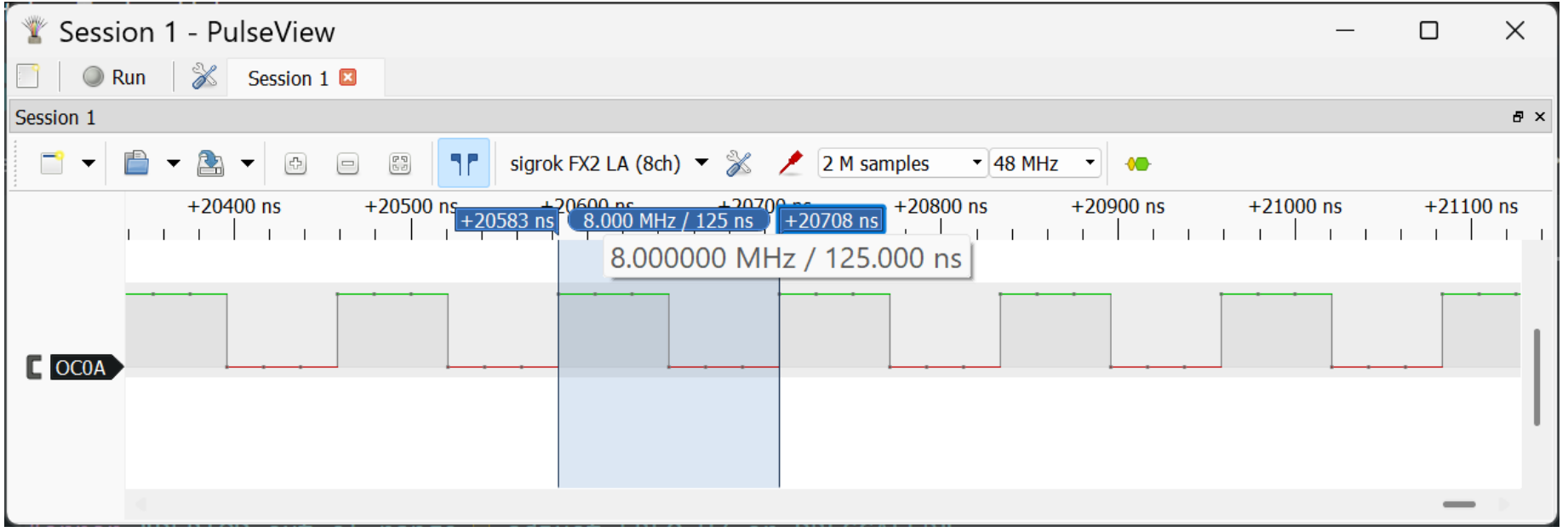
UART baudrate: 1000000

C Code Example 14: Timer2-based Pulse Signal Generator

```
#include <avr/io.h>
#define FREQ_HZ    (8000000UL)
#define PRESCALER  (1UL)
#define PERIOD     (F_CPU/(2UL * PRESCALER * FREQ_HZ))

#if (PERIOD == 0) || (PERIOD > 255)
  #error "PERIOD out of range – adjust FREQ_HZ or PRESCALER"
#endif

void timer0_init(void) {
  DDRD |= (1 << DDD6);  // OC0A / D6 as output
  TCCR0A = TCCR0B = 0; TCNT0 = 0;
  OCR0A  = (uint8_t)(PERIOD - 1);
  TCCR0A = (1<<COM0A0) | (1<<WGM01); // Toggle OC0A, CTC
#if (PRESCALER == 1)
  TCCR0B |= (1<<CS00);
#elif (PRESCALER == 8)
  TCCR0B |= (1<<CS01);
#elif (PRESCALER == 64)
  TCCR0B |= (1<<CS01) | (1<<CS00);
#elif (PRESCALER == 256)
  TCCR0B |= (1<<CS02);
#elif (PRESCALER == 1024)
  TCCR0B |= (1<<CS02) | (1<<CS00);
#else
  #error "Invalid PRESCALER value for Timer0. Allowed: 1,8,64,256,1024"
#endif
}
```

Timer2 is configured to operate in **CTC mode**, with output toggle on the **OC0A / D6 pin**.

USB Logic Analyzer: Waveform Capture



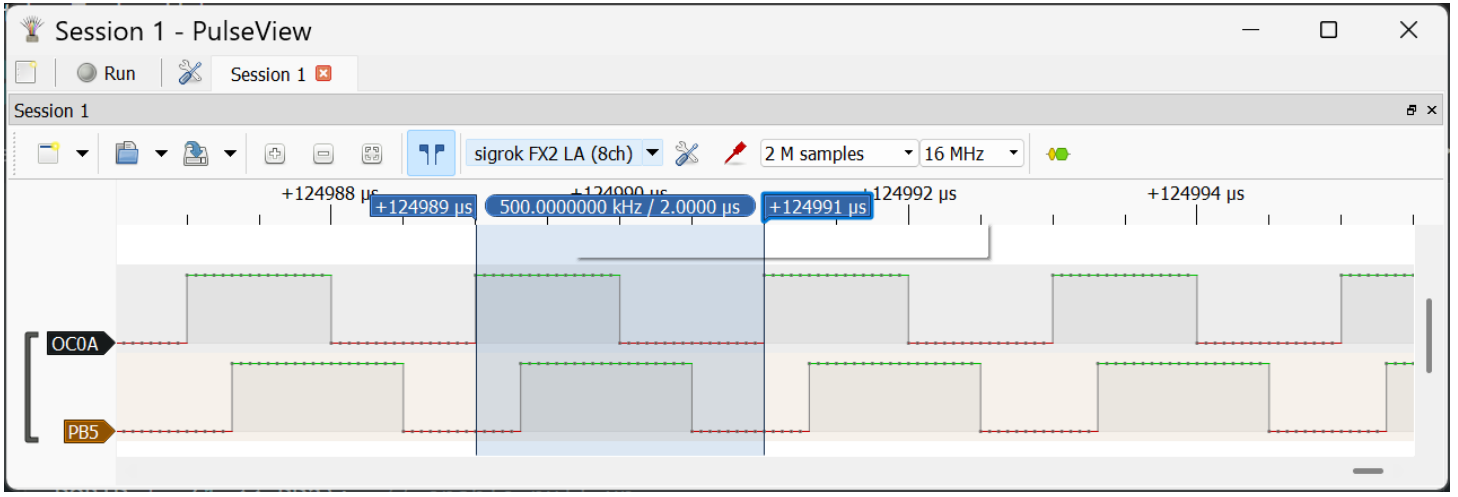
I/O Follower: Polling Method (No Interrupts)

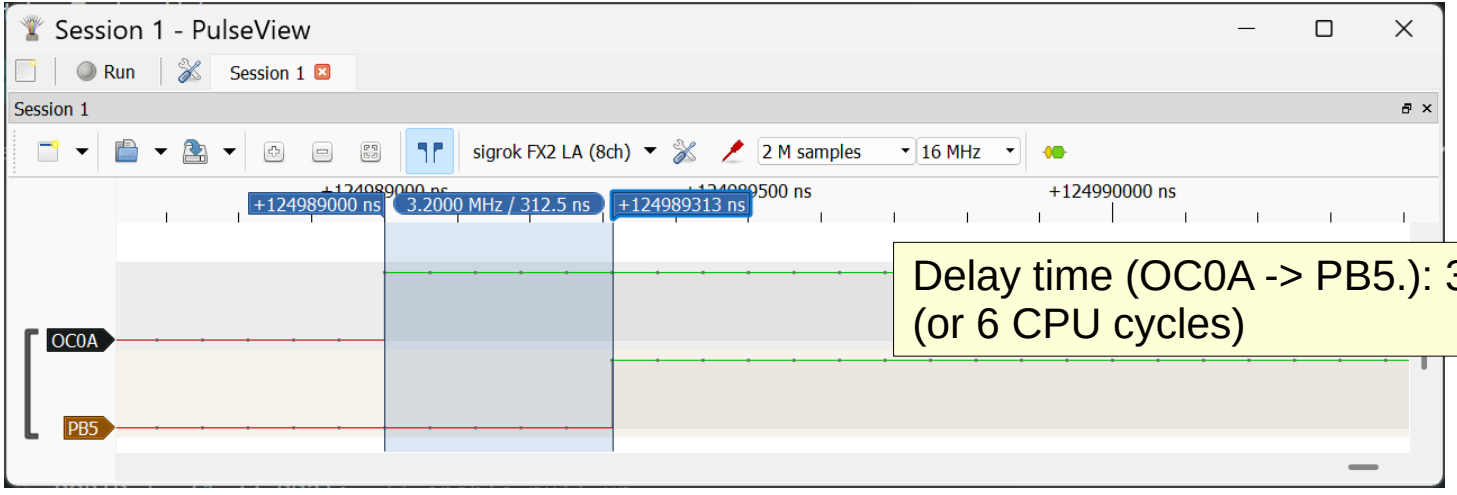
```
#include <avr/io.h>

extern void timer0_init(void);

int main(void) {
  DDRB |=  (1<<PB5);  // Output (D13)
  DDRD &= ~(1<<PD2);  // Input  (D2)
  PORTD |= (1<<PD2);  // Enable pull-up
  timer0_init();      // Initialize Timer0
  uint8_t last = 0;
  while (1) {
    if (PIND & (1<<PD2))
      PORTB |= (1<<PB5);  // Output High
    else
      PORTB &= ~(1<<PB5); // Output Low
  }
}
```

- This code uses **Timer0** to generate a periodic test pulse signal on the **OC0A** pin.
- The **OC0A** pin is connected to **PD2** (configured as a digital input) using a jumper wire. The internal pull-up resistor on **PD2** is enabled.
- Inside the main loop, the program continuously reads the logic level on **PD2** and sets the output pin **PB5 (D13)** to the same level. In this way, **PB5** follows the state of the Timer0-generated signal.

USB Logic Analyzer: Waveform Capture

Test frequency: 500kHz

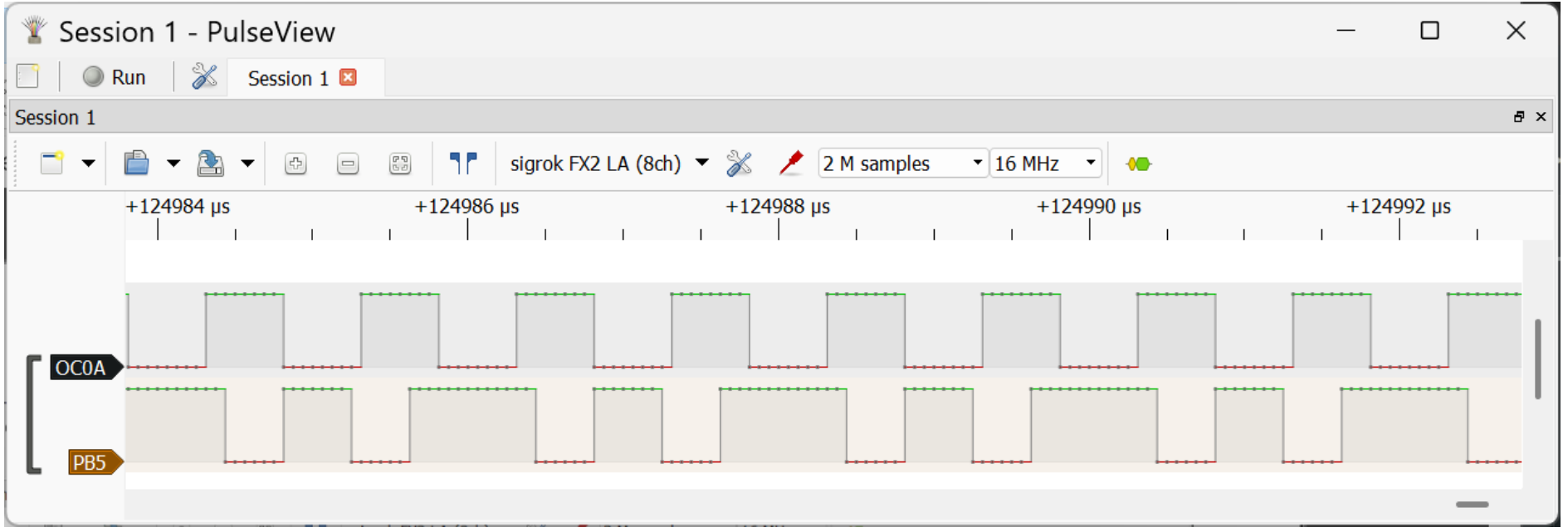




Delay time (OC0A -> PB5.): 312.5 ns
(or 6 CPU cycles)

USB Logic Analyzer: Waveform Capture

Test frequency: 1MHz

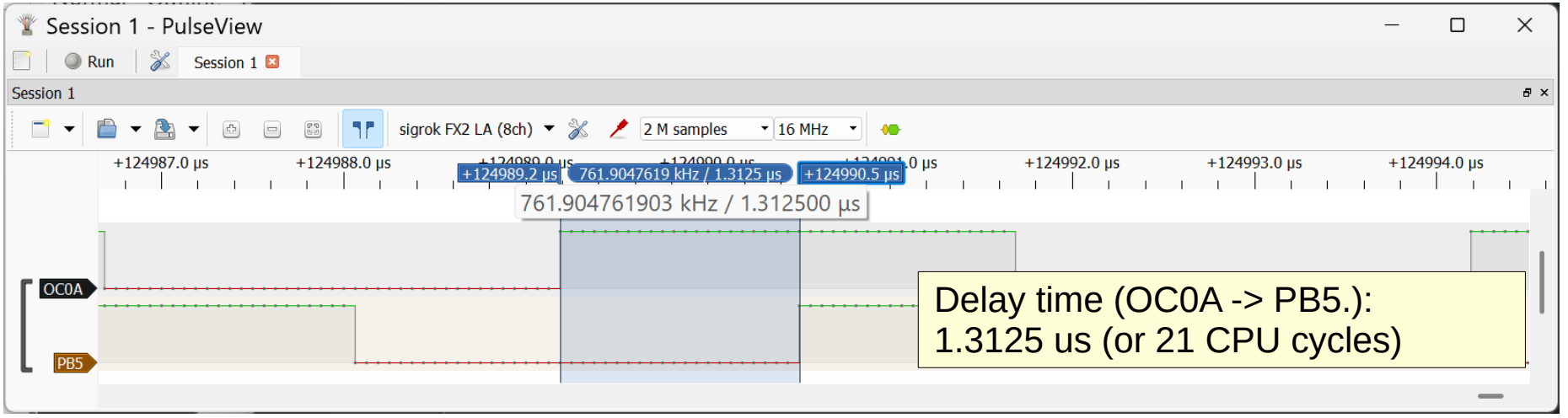


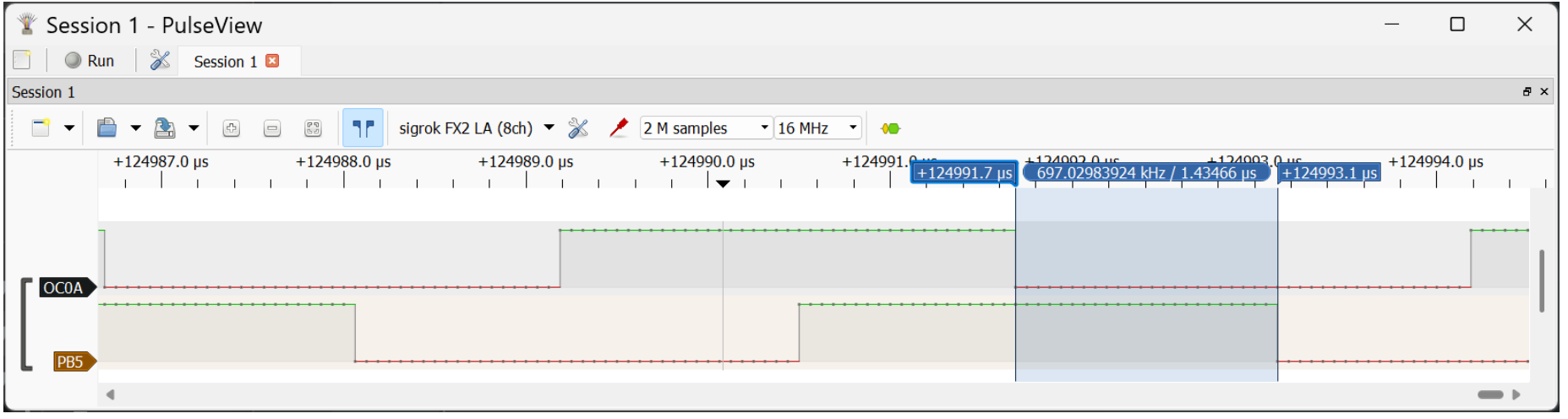
I/O Follower: Using an External Interrupt

```
#include <avr/io.h>
#include <avr/interrupt.h>

extern void timer0_init(void);

void gpio_init(void) {
  DDRB  |=  (1<<PB5); // PB5 as output
  DDRD  &= ~(1<<PD2); // PD2/INT0 as input
  PORTD |=  (1<<PD2); // Enable pull-up
}

void int0_init(void) {
  EICRA |=  (1<<ISC00); // Any change
  EICRA &= ~(1<<ISC01);
  EIMSK |=  (1<<INT0);  // Enable INT0
}
```

```
ISR(INT0_vect) {
  if (PIND & (1<<PD2))
    PORTB |= (1<<PB5);
  else
    PORTB &= ~(1<<PB5);
}

int main(void) {
  gpio_init();
  int0_init();
  timer0_init();
  sei();
  while (1) {}
}
```

USB Logic Analyzer: Waveform Capture



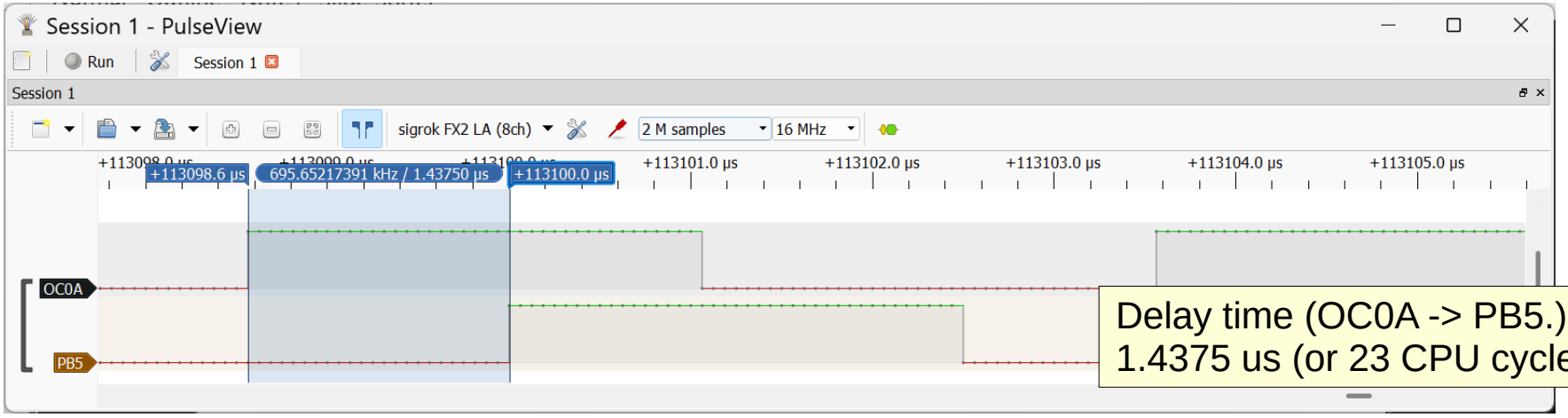


I/O Follower: Using an Pin Change Interrupt

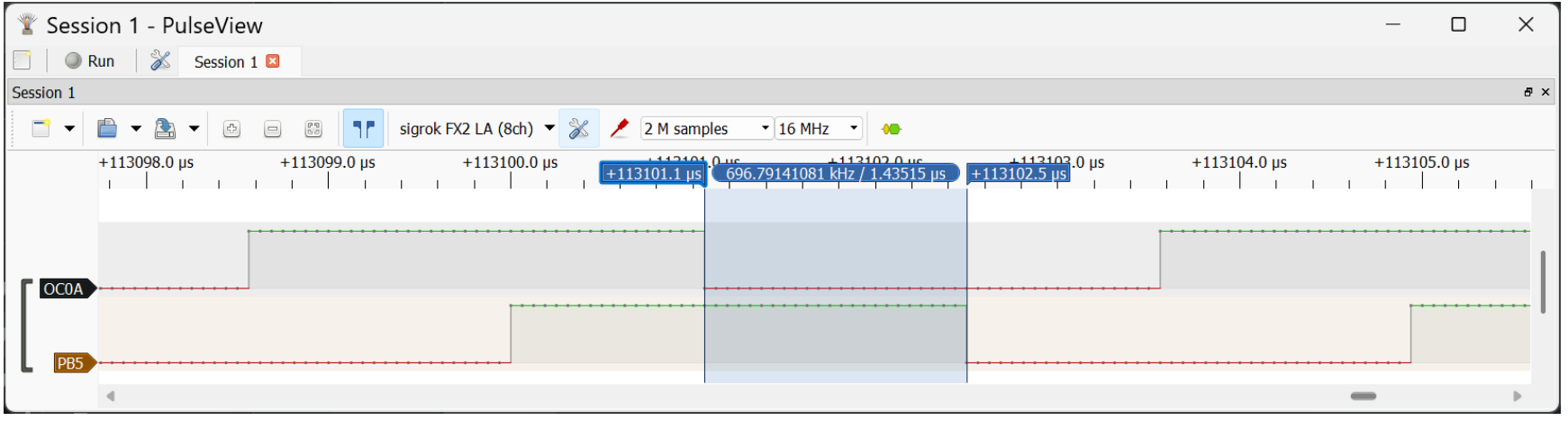
```
#include <avr/io.h>
#include <avr/interrupt.h>

extern void timer0_init(void);

void gpio_init(void) {
  DDRB |=  (1<<PB5); // PB5 as output
  DDRD &= ~(1<<PD2); // PD2 as Input (PCINT18)
  PORTD |= (1<<PD2); // Enable pull-up
}

void pcint_init(void) {
  PCICR  |= (1<<PCIE2); // Enable PCINT[23:16]
  PCMSK2 |= (1<<PCINT18); // Enable interrupt
}
```

```
ISR(PCINT2_vect) {
  if (PIND & (1<<PD2))
    PORTB |= (1<<PB5);
  else
    PORTB &= ~(1<<PB5);
}

int main(void) {
  gpio_init();
  pcint_init();
  timer0_init();
  sei();
  while (1){}
}
```

USB Logic Analyzer: Waveform Capture



Delay time (OC0A -> PB5.):
1.4375 us (or 23 CPU cycles)

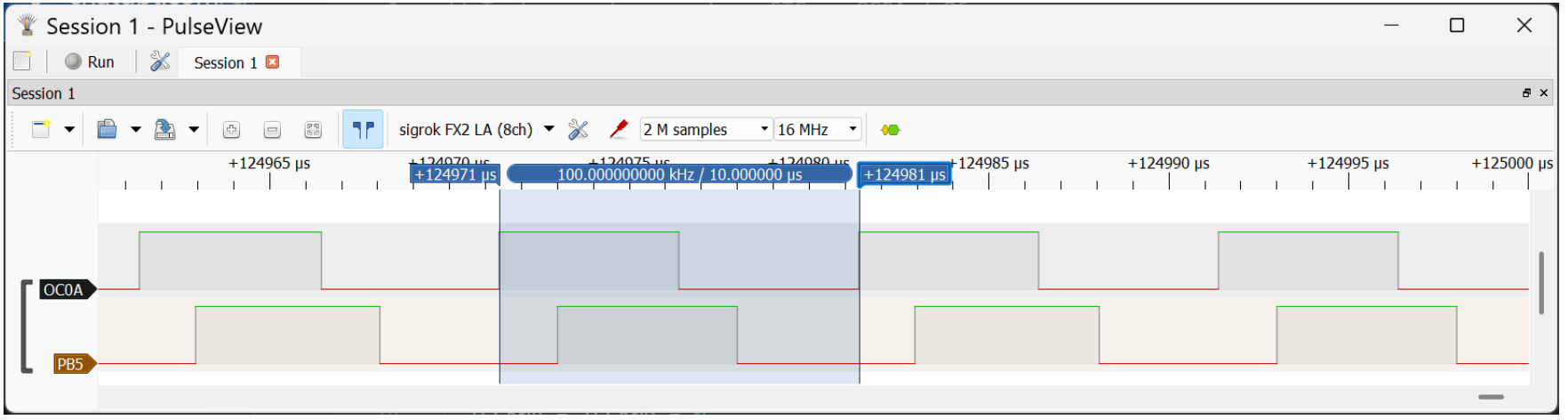


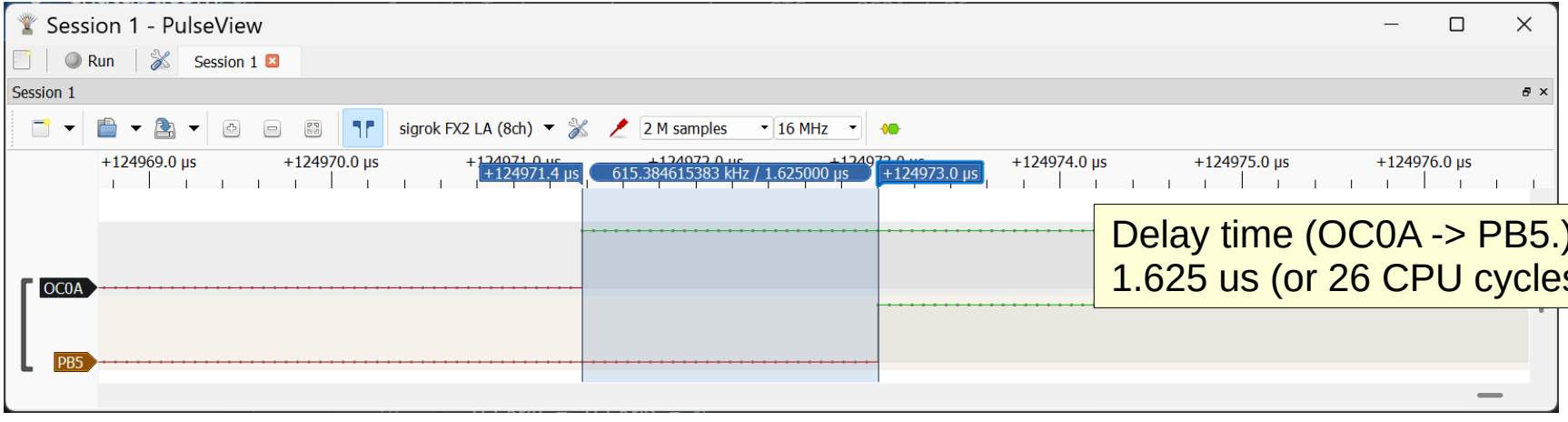
I/O Follower: Using Timer1 Input Capture Interrupt

```
#include <avr/io.h>
#include <avr/interrupt.h>

extern void timer0_init(void);

void gpio_init(void) {
  DDRB |=  (1<<PB5); // Output (D13)
  DDRB &= ~(1<<PB0); // ICP1 input (D8)
  PORTB |= (1<<PB0); // Enable pull-up
}

void timer1_icp_init(void) {
  TCCR1A = TCCR1B = 0;
  TCCR1B |= (1<<ICES1); // Rising edge
  TIMSK1 |= (1<<ICIE1); // Enable interrupt
  TCCR1B |= (1<<ICNC1); // Noise canceler
  TCCR1B  |= (1<<CS10);   // Start Timer1
}
```

```
ISR(TIMER1_CAPT_vect) {
  if (PINB & (1<<PB0))
    PORTB |= (1<<PB5);
  else
    PORTB &= ~(1<<PB5);
  TCCR1B ^= (1<<ICES1); // Switch edge
}

int main(void) {
  gpio_init();
  timer1_icp_init();
  timer0_init();
  sei();
  while (1) {}
}
```

USB Logic Analyzer: Waveform Capture





I/O Follower: Using Analog Comparator Interrupt

```
#include <avr/io.h>
#include <avr/interrupt.h>

extern void timer0_init(void);

void gpio_init(void) {
  DDRB |=  (1<<PB5); // PB5 (D13) as output
  DDRD &= ~(1<<PD6); // PD6 (AIN0) as input
}

void analog_comparator_init(void) {
  ACSR = 0;
  ACSR |= (1<<ACBG); // 1.1V bandgap
  ACSR |= (1<<ACIE); // Enable interrupt
}
```

```
ISR(ANALOG_COMP_vect) {
  PINB = (1<<PB5); // Toggle PB5
}

int main(void) {
  gpio_init();
  analog_comparator_init();
  timer0_init();
  sei();
  while (1) {}
}
```

USB Logic Analyzer: Waveform Capture